

Guías de cierre y rúbricas del examen suspenso

Proyecto Fin de Curso — Aplicaciones Distribuidas (ISR-701)

Universidad Técnica Estatal de Quevedo · 2026–2027 PPA · Séptimo semestre “A”
Software

Estado evaluado: domingo 13 de septiembre de 2026, 19:00 · Corte: viernes 18 de
septiembre de 2026, 23:55

Contenido y estado al 13/09/2026, 19:00

Equipo	Sistema	Repositorio	Compleitud	Nota provisional
FUVV	SCLI, Sistema de Control de Laboratorios Informáticos	gleiston-guerrero/ Entrega-final-del-PFC	64 %	3,37 / 10
ACC	TicketFold, soporte técnico distribuido para un proveedor de Internet	gleiston-guerrero/ PFC-SOPORTE-ISP	55 %	2,64 / 10
AGLS	TiendaTech, comercio electrónico distribuido (repositorio sin transferir)	JoseLozanoMorales/ TiendaTech	56 %	2,45 / 10
BCEL	AcadTrace, gestión académica con bitácora de auditoría encadenada	gleiston-guerrero/ acadtrace	40 %	2,65 / 10

La completitud es el promedio del estado efectivo de los veinte entregables exigidos por la guía de la Entrega 4, con la regla de dependencia. La nota provisional es la de la rúbrica del examen suspenso con el estado encontrado en el último commit anterior al 13/09/2026 a las 19:00: sólo puntúan los ítems que no estaban en Hecho, con pesos proporcionales a profundidad, complejidad, amplitud y tiempo de dedicación, y coeficientes Hecho 1,00 · Por modificar 0,40 · Por culminar 0,20 · Por iniciar 0,00. Cláusulas comunes: repositorio transferido y aceptado por el docente al corte (en caso contrario, examen no entregado) y carátula de una sola página subida al SGA por cada estudiante. Ing. Gleiston Cicerón Guerrero Ulloa, PhD.

Guía de cierre y rúbrica del examen suspenso

Equipo FUVV — SCLI, Sistema de Control de Laboratorios Informáticos

Identificación	
Asignatura	Aplicaciones Distribuidas (ISR-701) — 2026–2027 PPA
Paralelo	7.º “A” Software
Repositorio evaluado	gleiston-guerrero/Entrega-final-del-PFC
Cabecera evaluada	Rama feature/entrega-4, commit d0ef60b — 366 commits — último commit anterior al domingo 13/09/2026, 19:00; verificación sobre clon completo, 16 de septiembre de 2026
Integrantes	Farinango Guandinango Freddy Vladimir · Urbina Romero Isaías Abraham · Villamarín Cuenca Iván Andrés · Vinueza Sánchez Harold Nicolás
Rinden el examen suspenso	Urbina Romero Isaías Abraham · Villamarín Cuenca Iván Andrés
Compleitud del proyecto	64 %
Corte del examen suspenso	Viernes 18 de septiembre de 2026, 23:55 (Ecuador)
Docente	Ing. Gleiston Cicerón Guerrero Ulloa, PhD.

Cláusulas de entrega del examen suspenso (anulan la calificación si no se cumplen)

1. Repositorio transferido y aceptado. El repositorio debe estar transferido a la cuenta del docente (gleiston-guerrero) y la transferencia **aceptada** antes del corte. Si al viernes 18/09/2026 a las 23:55 el repositorio no figura bajo github.com/gleiston-guerrero/, el examen se toma como **no entregado** y su nota es cero, cualquiera que sea el trabajo realizado. Estado al 13/09: la transferencia se hizo la noche del 13/09 y al 16/09 el repositorio figura bajo [gleiston-guerrero/Entrega-final-del-PFC](https://github.com/gleiston-guerrero/Entrega-final-del-PFC); cumple si sigue así al corte. Además, al corte la rama `main` tenía un único commit del 15/08 y todo el trabajo estaba en `feature/entrega-4`. Todo lo que se califica se revisa en esa URL y en su rama principal; lo que quede en ramas sin fusionar o en cuentas personales no existe para esta evaluación.

2. Carátula en una sola página subida al SGA. Cada estudiante que rinde el examen sube al SGA un único PDF de **una sola página**, sin capturas de cuestionarios, calificaciones ni contenido adicional, con: universidad, facultad y carrera; asignatura, paralelo y periodo; “Examen suspenso — Proyecto Fin de Curso”; nombre completo del estudiante que la sube; equipo y nombre del sistema; integrantes del equipo; docente; URL canónica del repositorio (<https://github.com/gleiston-guerrero/Entrega-final-del-PFC>) escrita completa en una línea y como enlace activo; rama, etiqueta anotada de cierre y hash corto del commit final; fecha de entrega. Una carátula de más de una página, con una URL distinta de la canónica o que no se suba al SGA equivale a examen no entregado para ese estudiante.

1. Evaluación de los entregables

#	Entregable / indicador	Estado propio	Estado efectivo	%	Acción
1	Identificación del entregable y URL	Por modificar	Por modificar	75 %	Véase §1
2	Estructura del repositorio y archivos de raíz	Por modificar	Por modificar	70 %	Véase §2
3	Línea base congelada e integración a <code>main</code>	Por culminar	Por culminar	25 %	Véase §3
4	Backend en capas y SOLID	Por modificar	Por modificar	65 %	Véase §4
5	Patrones GoF y ADR	Por modificar	Por modificar	70 %	Véase §5
6	Aplicación web funcional	Por modificar	Por modificar	75 %	Véase §6
7	Calidad de la aplicación web	Por modificar	Por modificar	80 %	Véase §7
8	Aplicación móvil funcional	Por modificar	Por modificar	70 %	Véase §8

#	Entregable / indicador	Estado propio	Estado efectivo	%	Acción
9	Calidad de la aplicación móvil	Por modificar	Por modificar	60 %	Véase §9
10	Integración y contratos	Por modificar	Por modificar	60 %	Véase §10
11	Persistencia distribuida	Por modificar	Por modificar	75 %	Véase §11
12	Pirámide de pruebas	Por modificar	Por modificar	70 %	Véase §12
13	Pipeline CI/CD	Por modificar	Por modificar	70 %	Véase §13
14	Observabilidad	Por modificar	Por modificar	60 %	Véase §14
15	Protocolo y campaña experimental	Por modificar	Por modificar	60 %	Véase §15
16	Paquete de datos y reproducibilidad	Por modificar	Por modificar	60 %	Véase §16
17	Evaluación ISO/IEC 25010	Por modificar	Por modificar	55 %	Véase §17
18	Manuscrito, bibliografía y trazabilidad E1–E4	Por modificar	Por modificar	55 %	Véase §18
19	Amenazas a la validez y reflexión ética	Por modificar	Por modificar	80 %	Véase §19
20	Trabajo en equipo y autoría	Por modificar	Por modificar	55 %	Véase §20

Cómo leer las dos columnas de estado

El *estado propio* describe el artefacto en sí. El *estado efectivo* aplica la regla de dependencia: **ningún artefacto puede estar más completo que aquello de lo que se construye**. El backend en capas sostiene a los dos clientes; los clientes y el backend sostienen los contratos y la pirámide de pruebas; las pruebas sostienen el pipeline; el sistema instrumentado sostiene la campaña experimental; la campaña sostiene el paquete de datos; el paquete sostiene la evaluación ISO/IEC 25010; y el manuscrito, las amenazas a la validez y la línea base congelan y reportan todo lo anterior. Las actas, la declaración de uso de IA y los archivos de raíz no propagan: se declaran. Lo que se califica es el estado efectivo.

Lo que está hecho y no se recalifica

Ningún entregable está completo, pero hay piezas verificadas que no se tocan. Eficiencia (PI1) recalculada desde los datos crudos de `eficiencia_nominal_50u_5m`, repeticiones 2 a 9, sobre la fila `GET /api/v1/reservas`: p95 media 45,5 ms, IC95 % [27,30; 63,70]; p99 media 371,25 ms, IC95 % [66,91; 675,59], idénticas al documento y con umbrales fijados antes de medir. Campaña ARBITER: 123 de 130 corridas, manifiesto 315/315, censo HTTP de Esc-2 y Esc-3 idéntico al publicado y `analisis-e3.json` regenerado byte a byte. Manifiesto general 468/468 sobre los bytes del repositorio. Validador de contratos en tiempo de ejecución sobre los cinco servicios (12/44/71/79/224 operaciones). Instantánea documental E3 congelada (14 páginas, cero indefinidas). Manuscrito vivo de 33 páginas sin citas ni referencias indefinidas, 29 referencias y 14 DOI resueltos. SPA en TypeScript estricto con `tsc` y lint limpios y 280/280 pruebas; JWT en `sessionStorage` con expiración; serie temporal y calendario semanal sobre la API real. Móvil con `EncryptedSharedPreferences`, Room sin conexión, escáner QR con CameraX y ML Kit, unas 120 pruebas de ViewModel y APK con firma v1/v2/v3 de clave propia. Backend con 869 pruebas, Testcontainers con CockroachDB en tres servicios y JaCoCo ≥ 70 % con compuerta en los cinco; State, Facade, Repository y Factory Method genuinos; registros JSON con `trace_id` en los cinco servicios.

2. Rúbrica del examen suspenso

Cómo se califica el examen suspenso

Sólo se califican los ítems de esta tabla: lo que está en Hecho no vuelve a puntuar ni se puede perder. El peso de cada ítem sale de sumar cuatro estimaciones de lo que cuesta llevarlo a Hecho con la calidad exigida: **profundidad** (P, 1–5: de aplicar una instrucción a rehacer análisis, diseño experimental o arquitectura con criterio propio), **complejidad** técnica (C, 1–5: de trámite mecánico a coordinar varios servicios, datos y herramientas), **amplitud** (A, 1–5: de un archivo a varios módulos, clientes, flujos y documentos a la vez) y **tiempo** de dedicación (T, 1–5: menos de 2, de 2 a 6, de 6 a 15, de 15 a 30 y más de 30 horas-persona), normalizando a 100 dentro del equipo. Cada ítem recibe su peso multiplicado por el coeficiente del estado en que lo encuentre el viernes 18 de septiembre a las 23:55: **Hecho 1,00 · Por modificar 0,40 · Por culminar 0,20 · Por iniciar 0,00**. La nota del examen es la suma de puntos dividida entre diez. Un ítem sólo llega a Hecho si la comprobación descrita en “Cómo lo verifico” da el resultado exigido sobre el repositorio del docente; el mensaje de un commit, un archivo vacío o una cifra sin dato crudo que la respalde no cuentan. Las cláusulas de entrega de la primera página y la ausencia total de la aplicación web o de la móvil (factor 0,5) siguen vigentes sobre la nota.

§	Ítem que se califica	P	C	A	T	Pts	Peso %	Estado 13/09	Puntos
§1	URL canónica, nombre único del sistema y del equipo, autores completos en CITATION.cff	1	1	2	1	5	1,8	Por modificar	0,72
§2	LICENSE, .env.example con web y móvil, retirar el PDF de calificación, mapa del Listado 3 y .gitattributes de datos	1	2	2	1	6	2,2	Por modificar	0,88
§3	Integración a main por PR con revisión sustantiva y CI en verde, etiqueta anotada de cierre y CHANGELOG.md	2	2	3	2	9	3,3	Por culminar	0,66
§4a	Cero dependencias de application hacia infrastructure y presentation (228 imports en 56 archivos)	3	3	4	4	14	5,4	Por modificar	2,16
§4b	Dominio sin tipos Spring/JPA, paquetes heredados de reservas reubicados y reglas ArchUnit en mvn verify	2	2	3	3	10	3,7	Por modificar	1,48
§5a	Observer de alertas con dos o más suscriptores reales y Strategy con dos o más políticas de reserva seleccionables	3	3	3	3	12	4,4	Por culminar	0,88
§5b	Singleton del cliente Prometheus conforme, expiración por el patrón State y ADR-005/006 exactos	2	2	2	2	8	2,9	Por modificar	1,16
§6	Internacionalización de toda la SPA, /settings y /about para todo usuario, inicio real y tema oscuro coherente	1	2	4	3	10	3,7	Por modificar	1,48
§7	Cobertura web $\geq 70\%$ sobre todo src/ y E2E Playwright sin saltos, ejecutadas en verde	2	3	3	3	11	4,0	Por modificar	1,60
§8a	APK release contra el gateway real (sin 10.0.2.2 ni HTTP plano) y QR que abre la reserva rápida	2	2	2	2	8	2,9	Por modificar	1,16

§	Ítem que se califica	P	C	A	T	Pts	Peso %	Estado 13/09	Puntos
§8b	Notificaciones push FCM de extremo a extremo al crear incidentes, con evidencia de recepción	3	4	3	3	13	4,8	Por culminar	0,96
§9a	Pruebas móviles JUnit 5, cobertura ≥ 70 % con compuerta y E2E instrumentada login→lista→detalle	3	3	4	4	14	5,1	Por culminar	1,02
§9b	APK firmado generado desde main tras las pruebas y publicado de forma verificable	2	2	1	1	6	2,2	Por modificar	0,88
§10a	Contratos generados por los clientes reales (web y móvil) y verificados contra el proveedor arrancado, incluido auth	3	3	3	4	13	4,8	Por modificar	1,92
§10b	Validación de esquemas OpenAPI no circular como compuerta de publicación	2	3	2	2	9	3,3	Por modificar	1,32
§11	Experimento de caída de nodos reproducible, verificador de CI corregido y límite de licencia resuelto o justificado	3	3	2	3	11	4,0	Por modificar	1,60
§12	Informes de las cinco capas de pruebas como artefactos y carga nominal y rampa repetidas con el harness corregido	2	3	3	3	11	4,0	Por modificar	1,60
§13	Pipeline completo en verde en el commit final, lint en los cinco servicios y compuertas completas antes de publicar	2	3	3	2	10	3,7	Por modificar	1,48
§14a	Seis paneles con datos bajo carga, cuatro métricas nuevas en los servicios y traza distribuida exportada	2	3	3	3	11	4,0	Por modificar	1,60
§14b	Latencia extremo a extremo medida en el cliente móvil y propagada con traceparent	3	4	3	3	13	4,8	Por iniciar	0,00
§15a	Campaña correctiva de fiabilidad 10×1 h ejecutada y versionada, sin 401 inducidos por el instrumento	3	3	2	4	12	4,4	Por culminar	0,88
§15b	ARBITER: n suficiente en Esc-1, escenarios que ejerciten I4 e I5 y censo HTTP de todos los escenarios	4	3	3	3	13	4,8	Por modificar	1,92
§16	Manifiestos que verifican en cualquier clon y en CI, CSV canónico único y datos de toda ejecución citada	2	2	3	3	10	3,7	Por modificar	1,48
§17	ISO/IEC 25010 sobre la población válida: fiabilidad corregida, complejidad ciclomática, seguridad por endpoint y compatibilidad API 26/34	3	3	3	3	12	4,4	Por modificar	1,76
§18	Manuscrito: estructura de la Tabla 3, resúmenes ≥ 200 palabras, capturas exigidas, PDF versionado, instantánea E4 congelada y cifras canónicas	2	2	3	3	10	3,7	Por modificar	1,48
§19	Amenazas de sesión y licencia con mitigación real y reflexión ética $\geq \frac{1}{2}$ página	2	1	1	1	5	1,8	Por modificar	0,72
§20	Actas contemporáneas, PR revisados por otro integrante y declaración de uso de IA con alcance	1	1	2	2	6	2,2	Por modificar	0,88
Total							100		

Lectura de la rúbrica al 13/09/2026, 19:00

La columna *Estado* es el que encontré en el commit de corte; la columna *Puntos* es peso por coeficiente. Nota provisional del examen suspenso: **3,37 / 10,00**. Si cierran todos los ítems en Hecho, la nota es 10,00.

3. Trabajo exigido**§1 Identificación del entregable y URL**

Qué encontré: la URL del propietario anterior (ffarinangog2/Entrega-final-del-PFC) sigue en ocho archivos, entre ellos CITATION.cff, README.md, docs/main.tex (\RepoURL), docs/entrega-4/main.tex y docs/actas/README.md. El sistema tiene tres nombres (“Laboratorios e Infraestructura”, “Laboratorios e Informática”, “Laboratorios Informáticos”), el equipo aparece como “ForáCode” en la portada y como FUVV en el resto, y CITATION.cff lista un nombre y un apellido por autor, sin roles. **Cómo debe quedar:** <https://github.com/gleiston-guerrero/Entrega-final-del-PFC> en README, CITATION.cff y documentos vivos; un solo nombre de sistema y de equipo; nombres completos y roles de los cuatro integrantes. **Cómo lo verifico:** `git grep -n "ffarinangog2/Entrega" HEAD -- README.md CITATION.cff docs/main.tex` no devuelve nada.

§2 Estructura — licencia, variables, raíz y finales de línea

Qué encontré: no existe LICENSE; .env.example no tiene variables de la web ni del móvil; el commit de corte versiona un PDF con la calificación personal de un integrante en el cuestionario del SGA, ajeno al proyecto; faltan docs/api/openapi.yaml, protocolo-e3.md y boxplot_latencia.png del Listado 3 sin que el README explique la equivalencia; y .gitattributes sólo fija eol=lf para dos rutas, de modo que en un clon de Windows fallan 373 de 468 sumas del manifiesto general, 50 de 50 de evidencia-e3-canónica y la del APK. **Qué deben hacer:** añadir LICENSE; declarar VITE_API_BASE_URL y SCLI_API_BASE_URL en .env.example; retirar el PDF del cuestionario; añadir al README una tabla “Listado 3 → ruta real”; declarar experimentos/** -text y release/apk/SHA256SUMS.txt -text y renormalizar. **Cómo lo verifico:** `test -f LICENSE; git ls-files | grep -i cuestionario` vacío; en un clon con `core.autocrlf=true` y otro con `false`, `sha256sum -c` no imprime ningún fallo.

§3 Línea base — nada integrado ni congelado al corte

Qué encontré: al 13/09 a las 19:00 main tenía un único commit (859bc58, “Entrega anterior”, 15/08). El PR que integra feature/entrega-4 tiene 367 commits, 2.417 archivos y una sola aprobación de una frase genérica, sin un comentario en línea, emitida con el pipeline de ese commit en rojo; la fusión la hizo el propio autor a las 21:28, fuera de plazo y con dos commits que la aprobación no cubre. No hay ninguna etiqueta ni CHANGELOG.md. **Cómo debe quedar:** todo el trabajo en main mediante PR cuyo revisor, distinto del autor, deja comentarios sobre el código y aprueba con el pipeline en verde sobre el commit final; CHANGELOG.md con una entrada fechada por entrega (E1 a E4) y una del examen suspenso; etiqueta anotada de cierre sobre el último commit de main, creada **al final de todo**. **Cómo lo verifico:**

```
git cat-file -t v4.0.0          # debe imprimir: tag
git merge-base --is-ancestor v4.0.0 origin/main && git rev-parse v4.0.0~{commit} origin/main
git ls-remote --tags origin v4.0.0    # deben salir DOS líneas
```

y en la API de GitHub, por cada PR del suspenso, /reviews con un aprobador distinto del autor y /comments con al menos un comentario sobre el código.

§4 Backend en capas — la capa de aplicación depende de las externas

Qué encontré: 71 imports de `application` hacia `infrastructure` y 157 hacia `presentation`, en 56 archivos de cuatro servicios; por ejemplo `AdminUsuarioService` usa directamente `UsuarioAuthRepository` saltándose su propio puerto, y `NotificacionService` usa un repositorio JPA. Los diez puertos de académico importan `Page` y `Pageable` de Spring; `BloqueoAgendaRepositoryPort` expone una entidad con `@Entity`; reservas conserva los paquetes `entity`, `repository`, `client`, `mapper` y `security` fuera de las capas; y nada lo hace cumplir en CI. **Cómo debe quedar:** cero dependencias de `application` hacia `infrastructure` o `presentation`; dominio sin tipos de Spring ni de JPA; reglas `layeredArchitecture()` de ArchUnit por servicio ejecutadas en `mvn verify`. **Qué deben hacer:** mover los DTO a `application/dto` y mapear en el controlador; sustituir los repositorios JPA usados en aplicación por puertos; crear un tipo de paginación propio del dominio en académico (usuarios ya lo tiene) y un modelo de dominio para `BloqueoAgenda`. **Cómo lo verifico:**

```
grep -rEn "import .*\.(infrastructure|presentation)\." services/*/src/main/java --include=*.java | grep
↪ /application/ | wc -l # 0
grep -rEn "org\.springframework\.jakarta\.persistence" services/*/src/main/java --include=*.java | grep
↪ /domain/ | wc -l # 0
grep -rL "layeredArchitecture" services/*/src/test | wc -l
↪ # >= 4
```

§5 Patrones GoF — tres de los cinco exigidos no cumplen

Qué encontré: State, Facade, Repository y Factory Method son genuinos. El “Singleton del cliente Prometheus” (`HttpRequestsMetricsRegistry`) es un `@Component` con constructor público que asigna `instance = this`, envuelve `MeterRegistry` y está duplicado en usuarios; el cliente real, `PrometheusQueryClient`, es un bean normal. El Observer “de alertas” no existe: sólo hay un oyente de perfil que escribe en el log, y la palabra “alert” no aparece en el código de producción. La Strategy “de políticas de reserva” tiene una única implementación fija, y la estrategia real (`ArbitrajeStrategy S0–S4`) está desactivada en operación y no figura en el ADR. `SolicitudExpirationScheduler` cambia el estado con un `setEstado` que se salta el patrón State. ADR-006 usa el estado “Implementado”, ajeno al formato Nygard, y sus dos criterios cuantitativos no comparan ninguna cifra con Flutter o React Native. **Cómo debe quedar:** Observer de alertas o incidentes con dos o más suscriptores reales (notificación push y auditoría o métrica); dos o más políticas de reserva seleccionables en ejecución y probadas; singleton conforme (constructor privado o justificación explícita del singleton del contenedor) sin copias; expiración por el patrón State; ADR-005 con los conteos reales y ADR-006 en formato Nygard con una comparación cuantitativa entre alternativas y fuentes fechadas. **Cómo lo verifico:** `grep -rn "implements .*Strategy" services/reservas-solicitudes-service/src/main | wc -l ≥ 2` con su prueba de selección; dos o más clases suscriptoras del sujeto de alertas; lectura de ambos ADR.

§6 Aplicación web — internacionalización aparente

Qué encontré: las cinco rutas, la ruta protegida, el JWT con expiración, la serie temporal y el calendario funcionan, y `es.json` y `en.json` tienen las mismas 57 claves; pero sólo 3 de 38 componentes usan `t()` y quedan unos 283 textos fijos en español (inicio de sesión, calendario, monitoreo, menú, mensajes de error), de modo que al cambiar a inglés casi toda la aplicación sigue en español. `/settings` y `/about` exigen el rol de administrador; `/` sólo redirige a `/login`; el tema oscuro es un único bloque CSS frente a 233 colores fijos; la serie temporal sólo la ve el administrador. **Cómo debe quedar:** toda la interfaz traducida con fechas en el idioma activo; `/settings` y `/about` para cualquier usuario autenticado; `/` como página de inicio; tema oscuro coherente en todas las pantallas mediante variables CSS. **Cómo lo verifico:** la prueba E2E de idioma y tema, ampliada, cambia a inglés y comprueba textos en `/login`, `/main` y el calendario; `grep -rL useTranslation apps/web/src --include=*.tsx | grep -v test` sólo lista componentes sin texto visible.

§7 Calidad web — cobertura sobre un alcance favorable y E2E que no corrieron

Qué encontré: lint y tsc limpios, 280/280 pruebas y Dockerfile multi-stage. La cobertura declarada (ramas 73,69 %) se mide sólo sobre los archivos que importan las pruebas porque vite.config.ts no define coverage.include; con include: src/** las ramas caen a 68,94 % y el umbral falla, con LoginPage, ProfilePage, PlanificacionPage, router/index.tsx y App.tsx al 0 %. Esa cifra mal acotada es la que llega a la tabla de mantenibilidad. En Playwright, una prueba se salta con test.skip porque ningún flujo define sus variables, otra está excluida en la configuración, las seis restantes no tocan la serie temporal ni el calendario, y en el commit de corte ninguna se ejecutó porque falló el paso previo del job de integración. Hay credenciales escritas en auth.spec.ts. **Cómo lo verifico:**

```
cd apps/web && npm ci && npx vitest run --coverage --coverage.include='src/**/*.ts,tsx' # código de salida
↪ 0
grep -rn "test.skip" apps/web/e2e # vacío
```

y el job de integración del commit final en verde con el recuento de pruebas Playwright en su registro.

§8 Aplicación móvil — el APK de entrega no puede hablar con el sistema

Qué encontré: Kotlin con Compose y MVVM, API mínima 26 y objetivo 34, inicio de sesión por el gateway, listado y detalle con actualización por arrastre y caché Room, y QR real. Pero el .dex del APK versionado contiene http://10.0.2.2:8080/ (el CI no define SCLI_API_BASE_URL) y el tráfico HTTP sin cifrar sólo se permite en el manifiesto de depuración, así que en release queda bloqueado. Las notificaciones push no funcionan: no hay google-services.json, el APK no contiene google_app_id, el backend tiene FIREBASE_ENABLED=false y el propio README lo admite. Crear un incidente no notifica a ningún técnico (sólo hay una notificación local), y el detalle del laboratorio escaneado no lleva a la reserva rápida que pide la Tabla 1. **Cómo debe quedar:** APK de release contra el gateway por HTTPS o con network_security_config restringido; push FCM de extremo a extremo desde la creación del incidente hasta los técnicos con permiso, con evidencia versionada (identificador del mensaje enviado, registro del dispositivo y captura); acción “Reservar este laboratorio” tras escanear. **Cómo lo verifico:** unzip -p release/apk/*.apk 'classes*.dex' | strings | grep -c 10.0.2.2 da 0; los recursos del APK contienen google_app_id; la creación de incidentes llama al puerto de notificación.

§9 Calidad móvil — sin prueba extremo a extremo y con cobertura del 38 %

Qué encontré: 291 pruebas unitarias, unas 120 de ViewModels, todas con JUnit 4; las siete instrumentadas prueban pantallas con dobles, un DAO y una migración, y ninguna recorre la aplicación (actividad, inicio de sesión, navegación, red). Cobertura de líneas recalculada desde jacocoTestReport.xml: 1.733/4.520 = 38,34 %, sin compuerta. El APK versionado se subió a mano el 12/09; su firma es válida con clave propia, pero nada lo vincula con el artefacto del CI; el job firmado sólo corre al empujar a feature/entrega-4 (en el PR salió skipped) y no depende de las pruebas. **Cómo debe quedar:** ViewModels con JUnit 5; cobertura ≥70 % con jacocoTestCoverageVerification; al menos una E2E instrumentada con createAndroidComposeRule<MainActivity> que recorra inicio de sesión, lista, detalle y modo sin conexión contra un servidor simulado; APK firmado generado desde main después de las pruebas y publicado de forma verificable (Release con su suma citada en el README o atestación de procedencia). **Cómo lo verifico:** grep -rl org.junit.jupiter apps/mobile/app/src/test | wc -l; grep -rn "createAndroidComposeRule<MainActivity>" apps/mobile/app/src/androidTest; la suma publicada coincide con sha256sum del APK descargado del CI.

§10 Contratos — escritos a mano y verificados sin el proveedor real

Qué encontré: los contratos consumidores están escritos a mano en Java dentro de `tests/contract`; no se generan desde la web (no hay `pact-js`) ni desde el móvil, y ya están desfasados: la app pide `GET api/v1/incidentes?tamano=100` y el contrato define ese GET sin parámetros. La verificación del proveedor usa `standaloneSetup` con servicios simulados, sin contexto de Spring ni seguridad, y `auth-service` no tiene prueba de proveedor, así que sus contratos nunca se verifican. El validador en tiempo de ejecución es una mejora real, pero sólo compara método y ruta, y el job `validate-openapi-contracts` sigue ejecutando el script que compara el JSON con su propio extractor por expresiones regulares, lo cual es circular. Además esos cambios rompieron el pipeline (§13). **Cómo debe quedar:** contratos generados por los clientes reales (`pact-js` sobre la capa de API web; pruebas consumidoras sobre las interfaces Retrofit), verificados contra el proveedor arrancado (`@SpringBootTest(webEnvironment = RANDOM_PORT)`) en cada servicio consumido, incluido `auth`, y validación de esquemas contra `/v3/api-docs` como compuerta de la publicación. **Cómo lo verifico:** `grep -n pact apps/web/package.json apps/mobile/app/build.gradle.kts; grep -rn standaloneSetup` en las pruebas de proveedor devuelve vacío; existe la prueba de proveedor de `auth`; job de verificación en verde en el commit final.

§11 Persistencia distribuida — tolerancia a fallos sin experimento reproducible

Qué encontré: tres nodos CockroachDB con TLS, `num_replicas=3`, Flyway en cuatro servicios y un script de CI que detiene un nodo (en verde). Pero la evidencia de tolerancia a fallos son capturas del 28/07 de otra topología (`--insecure`) más un vídeo de 24,6 MB, con latencias de una sola medición dominadas por el arranque del cliente, y cita una figura que no existe. Con el nodo caído el script sólo ejecuta `SELECT 1` y compara la versión de migración como texto, de modo que nunca verifica V10–V14. `Auth`, usuarios y académico corren en nodo único. Y el registro del servicio de reservas de la campaña de fiabilidad muestra “No license installed. The maximum number of concurrently open transactions has been reached”, que es el origen de los HTTP 500 de las campañas. **Cómo debe quedar:** carpeta `experimentos/crdb-failover/` con script, estado del clúster antes, durante y después, lecturas y escrituras con marca UTC, cinco o más repeticiones con uno y dos nodos caídos y manifiesto; verificador de CI que lea y escriba en tablas replicadas con el nodo caído y compare la versión de migración como entero; límite de licencia resuelto antes de volver a medir, o justificado en un ADR junto con la decisión de nodos únicos.

§12 Pirámide de pruebas — capas escritas pero no ejercitadas al corte

Qué encontré: 869 pruebas de backend, Testcontainers con CockroachDB en tres servicios y el `locustfile` ya renueva el testigo. Pero en el commit de corte falló el job de integración, de modo que Playwright y el *smoke* de Locust no se ejecutaron; no hay E2E móvil (§9); las diez corridas nominales y la única rampa son del 30/08, anteriores a la corrección del testigo, y la rampa registra `status: failed`; el CI no publica los informes de Playwright ni de JaCoCo; académico prueba integración con H2 y Flyway desactivado; Testcontainers usa CockroachDB v24.3.5 y el compose v26.2.0. **Cómo debe quedar:** las cinco capas en verde en el commit final, cada una con su informe como artefacto descargable; carga nominal (50 usuarios, 5 minutos) y rampa (0 a 200 en 10 minutos, tres o más corridas) repetidas con el *harness* corregido y sus datos crudos versionados; integración de académico sobre CockroachDB con migraciones reales y la misma versión que el compose.

§13 Pipeline CI/CD — en rojo en el commit de corte

Qué encontré: los siete jobs existen en un DAG coherente, sin `continue-on-error` ni pruebas saltadas, con sonda de salud y seis imágenes etiquetadas por commit. Pero hubo 11 commits consecutivos en rojo hasta el de corte, que no tiene imágenes publicadas. La causa, reproducida

en local: build-helper-maven-plugin añade fuentes de ../../tests/openapi-runtime, fuera del contexto de Docker, y jackson-databind tiene alcance test. El lint del backend sólo corre en académico con seis reglas; ESLint admite advertencias; validate-openapi-contracts y el job del APK firmado no bloquean la publicación de imágenes; el despliegue nunca se ejecutó. **Cómo debe quedar:** ejecución completa en verde sobre el commit etiquetado, con sus imágenes en GHCR; Checkstyle con failOnViolation en los cinco servicios; eslint . --max-warnings 0; todas las validaciones en los needs de build-images. **Cómo lo verifico:**

```
curl -s "https://api.github.com/repos/gleiston-guerrero/Entrega-final-del-PFC/actions/runs?head_sha=$(git
↪ rev-parse v4.0.0~{commit})"
grep -c "checkstyle:check" .github/workflows/ci-cd.yml # >= 5
```

§14 Observabilidad — la mitad del tablero vacío

Qué encontré: registros JSON con trace_id en los cinco servicios, app_business_events_total en flujos reales, app_active_sessions en auth, Collector hacia Tempo y Loki, y un tablero versionado con seis paneles. Pero la captura del propio equipo, tomada bajo carga, muestra “No data” en tres: disponibilidad del clúster (el scrape no declara scheme: https ni tls_config y el clúster usa certificados), recursos por contenedor (cAdvisor sin privilegios) y latencia desde el móvil, que en realidad es un indicador del gateway y tampoco tiene datos. La app no mide ni exporta latencia; su TraceId.kt es un UUID por proceso sin traceparent. http_requests_total sólo existe en dos servicios, http_request_duration_seconds es una regla de grabación sólo para reservas, el tablero no usa ninguna de las cuatro métricas nuevas y no hay una traza distribuida exportada. **Cómo debe quedar:** los seis paneles con series bajo carga Locust, con captura y respuestas query_range versionadas en ops/evidencia/; interceptor en el cliente móvil que mida la duración de cada petición, genere traceparent y exporte la métrica por OTLP; las cuatro métricas en todos los servicios que atienden tráfico y usadas en el tablero; una traza del móvil a CockroachDB exportada y correlacionada con su registro en Loki.

§15 Campaña experimental — la corrección de fiabilidad no se ejecutó

Qué encontré: PI1 quedó bien corregida (véase el recuadro de lo hecho). Fiabilidad no: el protocolo correctivo registra diez repeticiones de una hora con renovación de sesión, pero resultados/iso25010-correctiva.csv tiene sus diez filas vacías; la única repetición correctiva que el cierre declara “válida” y los tres intentos de la segunda no están en el repositorio (el documento lo admite), y lo único versionado es un smoke de 25 minutos. La campaña que sostiene PI2 sigue con 75,05 % de respuestas 401 inducidas por el instrumento, marcada valida=si. En ARBITER, Esc-1 perdió 7 de 10 corridas ($n = 3$ frente a $r = 10$); los invariantes I4 e I5 quedan NOT_OBSERVED en 96 de 96 casos, de modo que el oráculo sólo evalúa tres de cinco; y el censo HTTP omite Esc-4, donde S4 tuvo 351 de 1.600 errores (21,94 %) frente a 0 de S3, porque auditar_http_arbiter.py fija COMPARISON_SCENARIOS=("esc2", "esc3"). Seguridad, mantenibilidad y compatibilidad usan $r = 3$. **Cómo debe quedar:** campaña correctiva de fiabilidad 10×1 h ejecutada después de resolver el límite de transacciones (§11), con datos crudos, metadatos y sumas de cada repetición versionados; ARBITER con repeticiones suficientes en Esc-1 (o exclusión declarada en el protocolo antes de ejecutar), corridas de cancelación y acceso que ejerciten I4 e I5, y censo HTTP de todos los escenarios. **Cómo lo verifico:**

```
python -c "import csv;print(sum(1 for r in
↪ csv.DictReader(open('experimentos/resultados/iso25010-correctiva.csv')) if r['total_requests']))" # 10
grep -rho NOT_OBSERVED experimentos/resultados/arbiter/campaign/analysis/oracle | wc -l
grep -n COMPARISON_SCENARIOS experimentos/*.py
```

§16 Paquete de datos — sumas que no verifican y dos juegos de percentiles

Qué encontré: el manifiesto general verifica 468/468 sobre los bytes del repositorio y ARBITER 315/315, pero evidencia-e3-canonica/MANIFEST-SHA256.txt no verifica en ningún modo de checkout

(falla 43 de 50 sobre los bytes: 43 sumas se calcularon sobre la versión CRLF), el *smoke* de fiabilidad falla 3 de 7 y ningún job del CI verifica los manifiestos. No hay un juego canónico: *iso25010.csv* guarda los percentiles de la fila agregada (r1: p95 110, p99 2.200 ms) y el manuscrito publica los de la fila GET (r1: 87 y 1.000 ms), que no están en ningún CSV procesado. **Cómo debe quedar:** manifiestos regenerados sobre los bytes versionados y protegidos por *.gitattributes*; un paso del CI que ejecute `sha256sum -c` para cada manifiesto; un único CSV canónico con exactamente los valores que publica el documento; datos crudos de toda ejecución que el documento cite. **Cómo lo verifico** (en un clon limpio, en Linux y en Windows):

```
sha256sum -c experimentos/resultados/SHA256SUMS | grep -vc ': OK$'
↪ # 0
(cd experimentos/resultados/evidencia-e3-canonica && sha256sum -c MANIFEST-SHA256.txt | grep -vc ': OK$')
↪ # 0
(cd experimentos/evidencia-e2/smoke-refresh-25m && sha256sum -c SHA256SUMS.txt | grep -vc ': OK$')
↪ # 0
```

§17 Evaluación ISO/IEC 25010 — una respuesta afirmativa sobre una población inválida

Qué encontré: la tasa de 5xx publicada (0,061 %, IC95 % [0,032; 0,091]) se calcula sobre una población donde el 75 % son 401 que no llegan al servicio; con el denominador sin 401, recalculado desde *locust_failures.csv*, es 0,246 % [0,128; 0,364], cuatro veces más, y en el *smoke* con sesión renovada llega a 0,60 % (219 HTTP 500 en 36.331 GET), cerca del umbral del 1 %. Los p95/p99 de fiabilidad usan la fila agregada, con una regla de población distinta de la de P11. Mantenibilidad no mide la complejidad ciclomática que exige la Tabla 2 y usa la cobertura web mal acotada (§7). Seguridad cuenta 7 decisiones × 3 en vez del porcentaje de endpoints con verificación JWT sobre las 224 operaciones del gateway. Compatibilidad móvil “API 26+” sólo se probó en un emulador API 29. **Cómo debe quedar:** las cinco características con la métrica de la Tabla 2, sobre la población que declara el protocolo: fiabilidad desde la campaña correctiva de §15; mantenibilidad con cobertura de todo el código y complejidad ciclomática media por componente (PMD, Checkstyle o *lizard*); seguridad como porcentaje de endpoints protegidos del inventario OpenAPI con una prueba 401 por endpoint; compatibilidad móvil en API 26 y 34. Las métricas deterministas se reportan como medición única, sin intervalo.

§18 Manuscrito — dos introducciones y conclusiones que no son las finales

Qué encontré: compila a 33 páginas sin indefinidas y la bibliografía cumple. Pero no hay PDF versionado de la fuente oficial; *docs/entrega-4*, presentado como instantánea, incluye las secciones vivas, compila con una figura ausente (*panel-monitoreo.png*) y una referencia indefinida (*sec:contratos-openapi*), y su PDF es del 05/09. Hay dos secciones “Introducción”; las secciones de E4 se insertan antes de la discusión y las conclusiones de E3, de modo que el documento termina con las conclusiones de E3; la tabla de módulos de E3 sigue diciendo “Métricas Prometheus: No iniciado” y “Protocolo: sin repeticiones ejecutadas”; el resumen dice que las evidencias “aún deban ejecutarse”, no da ningún resultado y tiene 154 palabras en español y 97 en inglés. No hay capturas de las cinco rutas web, de la app móvil ni de una traza (las “figuras” de las aplicaciones son bloques de texto ASCII). **Cómo debe quedar:** un documento acumulativo con la estructura de la Tabla 3 de la guía (una introducción; discusión y conclusiones finales que integren E1–E4), resúmenes de al menos 200 palabras con los resultados, capturas reales de las cinco rutas en español e inglés, de la app y de una traza en Tempo; cifras idénticas al CSV canónico (§16–§17) y compilado *después* de la última regeneración de resultados; *docs/main.pdf* versionado e igual a la compilación de su fuente; *docs/entrega-4* congelado con copia propia de secciones y figuras, como ya se hizo con *docs/entrega-3*. **Cómo lo verifico:**

```
cd docs && latexmk -pdf -interaction=nonstopmode main.tex && grep -c "undefined\|not found" main.log #
↪ 0
cd entrega-4 && latexmk -pdf -interaction=nonstopmode main.tex && grep -c "undefined\|not found" main.log #
↪ 0
```

```
grep -rc '\\section{Introducci' ../main.tex secciones/*.tex
↪ total 1
```

#

Además extraigo el texto del PDF y busco cada valor de la tabla ISO en el CSV canónico.

§19 Amenazas a la validez y reflexión ética

Qué encontré: tres amenazas internas, dos externas y tres de constructo con el formato amenaza, efecto, mitigación y riesgo. Dos “mitigaciones” internas sólo informan (“se conservan y publican todos los intentos”, “se reporta separada”), y faltan las dos amenazas que más afectan a los resultados: el testigo no renovado (75 % de 401) y el límite de transacciones de CockroachDB sin licencia, causa de los HTTP 500. La reflexión ética cita el Código de Ética de ACM pero tiene 342 palabras, cerca de un tercio de página. **Cómo debe quedar:** ambas amenazas añadidas con una mitigación que las reduzca (campaña correctiva, licencia o ajuste del límite); las dos mitigaciones reescritas como control, repetición o medición adicional; reflexión de al menos media página vinculada a principios numerados del código (1.2, 1.6, 2.5, 3.1) aplicados a los datos de reservas y monitoreo.

§20 Trabajo en equipo y autoría

Qué encontré: las actas se reetiquetaron honestamente como “registros retrospectivos” escritos el 12/09 que “no demuestran reuniones”: no existe ningún acta contemporánea. La revisión cruzada se reduce a la aprobación descrita en §3. La declaración de uso de IA es genérica y no dice en qué artefactos se usó. Urbina Romero tiene 73 commits con aporte real en web, usuarios, móvil, Observer y ADR-006; Villamarín Cuenca tiene 91, pero la mitad de sus líneas son JSON de OpenAPI generados y, entre el 12/09 y el corte, un solo cambio sustantivo. **Cómo debe quedar:** un acta por reunión del periodo del suspenso, versionada el mismo día; cada ítem de esta rúbrica entregado en un PR cuyo autor es uno de los dos integrantes que rinden el examen y cuyo revisor es otro integrante, con comentarios sobre el código; declaración de IA con herramienta, tareas y cómo se verificó lo producido. El aporte de cada estudiante que rinde el examen se acredita sólo por sus propios commits y PR sobre los ítems de esta guía. **Cómo lo verifico:** `git log --format='%h%cI%aN' -- docs/actas/` frente a la fecha interna de cada acta; `git shortlog -sn --no-merges d0ef60b..v4.0.0; /pulls?state=all, /reviews y /comments` de la API.

4. Orden de ejecución

§13 pipeline en verde → §2 y §16 finales de línea, manifiestos y compuerta → §11 límite de transacciones → §15 campañas → §17 ISO/IEC 25010 y CSV canónico → §4 capas y ArchUnit → §5 patrones → en paralelo §6–§7 web y §10 contratos (Villamarín Cuenca) y §8–§9 y §14 móvil y observabilidad (Urbina Romero) → §12 pirámide → §18 manuscrito y §19 amenazas con las cifras definitivas → §1 URL → §20 y §3 PR revisados, fusión a `main`, `CHANGELOG`, manifiestos finales y, lo último, la etiqueta anotada.

5. Lista de verificación antes de entregar

```
git ls-remote https://github.com/gleiston-guerrero/Entrega-final-del-PFC HEAD # repositorio transferido y
↪ accesible
git cat-file -t v4.0.0 && git merge-base --is-ancestor v4.0.0 origin/main && test -f CHANGELOG.md -a -f LICENSE
sha256sum -c experimentos/resultados/SHA256SUMS | grep -vc ': OK$'
python experimentos/analizar_iso25010.py experimentos/resultados/iso25010-correctiva.csv
grep -rEn "import .*\.(infrastructure|presentation)\." services/*/src/main/java --include=*.java | grep -c
↪ /application/
cd apps/web && npx vitest run --coverage --coverage.include='src/**/*.ts,tsx'
cd docs && latexmk -pdf main.tex && grep -c "undefined\|not found" main.log
```

Documento de uso académico. Aplicaciones Distribuidas (ISR-701), UTEQ, 2026–2027 PPA. Verificación ejecutada sobre clon completo del repositorio declarado, con el estado al domingo 13 de septiembre de 2026 a las 19:00, el 16 de septiembre de 2026. Ing. Gleiston Cicerón Guerrero Ulloa, PhD.

Guía de cierre y rúbrica del examen suspenso

Equipo ACC — TicketFold, soporte técnico distribuido para un proveedor de Internet

Identificación	
Asignatura	Aplicaciones Distribuidas (ISR-701) — 2026–2027 PPA
Paralelo	7.º “A” Software
Repositorio evaluado	gleiston-guerrero/PFC-SOPORTE-ISP
Cabecera evaluada	Rama main, commit 964fa6e — 198 commits — último commit anterior al domingo 13/09/2026, 19:00; verificación sobre clon completo, 16 de septiembre de 2026
Integrantes	Pacheco Cárdenas Cristhian Daniel · Carpio Mendoza Carlos José · Álvarez Párraga Jeremy Alexis · Cando Moreno Robinson Rodrigo
Rinden el examen suspenso	Álvarez Párraga Jeremy Alexis
Compleitud del proyecto	55 %
Corte del examen suspenso	Viernes 18 de septiembre de 2026, 23:55 (Ecuador)
Docente	Ing. Gleiston Cicerón Guerrero Ulloa, PhD.

Cláusulas de entrega del examen suspenso (anulan la calificación si no se cumplen)

1. Repositorio transferido y aceptado. El repositorio debe estar transferido a la cuenta del docente (gleiston-guerrero) y la transferencia **aceptada** antes del corte. Si al viernes 18/09/2026 a las 23:55 el repositorio no figura bajo github.com/gleiston-guerrero/, el examen se toma como **no entregado** y su nota es cero, cualquiera que sea el trabajo realizado. Estado al 13/09: al 16/09 el repositorio figura bajo [gleiston-guerrero/PFC-SOPORTE-ISP](https://github.com/gleiston-guerrero/PFC-SOPORTE-ISP); cumple si sigue así al corte. Todo lo que se califica se revisa en esa URL y en su rama principal; lo que quede en ramas sin fusionar o en cuentas personales no existe para esta evaluación.

2. Carátula en una sola página subida al SGA. Cada estudiante que rinde el examen sube al SGA un único PDF de **una sola página**, sin capturas de cuestionarios, calificaciones ni contenido adicional, con: universidad, facultad y carrera; asignatura, paralelo y periodo; “Examen suspenso — Proyecto Fin de Curso”; nombre completo del estudiante que la sube; equipo y nombre del sistema; integrantes del equipo; docente; URL canónica del repositorio (<https://github.com/gleiston-guerrero/PFC-SOPORTE-ISP>) escrita completa en una línea y como enlace activo; rama, etiqueta anotada de cierre y hash corto del commit final; fecha de entrega. Una carátula de más de una página, con una URL distinta de la canónica o que no se suba al SGA equivale a examen no entregado para ese estudiante.

1. Evaluación de los entregables

#	Entregable / indicador	Estado propio	Estado efectivo	%	Acción
1	Identificación del entregable y URL	Por modificar	Por modificar	75 %	Véase §1
2	Estructura del repositorio y archivos de raíz	Por modificar	Por modificar	70 %	Véase §2
3	Línea base congelada e integración	Por culminar	Por culminar	25 %	Véase §3
4	Backend en capas y SOLID	Por modificar	Por modificar	65 %	Véase §4
5	Patrones GoF y ADR	Por modificar	Por modificar	70 %	Véase §5
6	Aplicación web funcional	Por modificar	Por modificar	75 %	Véase §6
7	Calidad de la aplicación web	Por modificar	Por modificar	75 %	Véase §7
8	Aplicación móvil funcional	Por modificar	Por modificar	65 %	Véase §8
9	Calidad de la aplicación móvil	Por modificar	Por modificar	55 %	Véase §9
10	Integración y contratos	Por culminar	Por culminar	45 %	Véase §10
11	Persistencia distribuida	Por modificar	Por modificar	70 %	Véase §11
12	Pirámide de pruebas	Por culminar	Por culminar	40 %	Véase §12

#	Entregable / indicador	Estado propio	Estado efectivo	%	Acción
13	Pipeline CI/CD	Hecho	Por modificar	85 %	Véase §13
14	Observabilidad	Por modificar	Por modificar	65 %	Véase §14
15	Protocolo y campaña experimental	Por culminar	Por culminar	35 %	Véase §15
16	Paquete de datos y reproducibilidad	Por modificar	Por culminar	35 %	Véase §16
17	Evaluación ISO/IEC 25010	Por culminar	Por culminar	30 %	Véase §17
18	Manuscrito, bibliografía y trazabilidad E1–E4	Por modificar	Por culminar	35 %	Véase §18
19	Amenazas a la validez y reflexión ética	Por modificar	Por modificar	60 %	Véase §19
20	Trabajo en equipo y autoría	Por culminar	Por culminar	25 %	Véase §20

Cómo leer las dos columnas de estado

El *estado propio* describe el artefacto en sí. El *estado efectivo* aplica la regla de dependencia: **ningún artefacto puede estar más completo que aquello de lo que se construye**. El backend en capas sostiene a los dos clientes; los clientes y el backend sostienen los contratos y la pirámide de pruebas; las pruebas sostienen el pipeline; el sistema instrumentado sostiene la campaña experimental; la campaña sostiene el paquete de datos; el paquete sostiene la evaluación ISO/IEC 25010; y el manuscrito, las amenazas a la validez y la línea base congelan y reportan todo lo anterior. Las actas, la declaración de uso de IA y los archivos de raíz no propagan: se declaran. Lo que se califica es el estado efectivo.

Lo que está hecho y no se recalifica

El pipeline es el más completo del curso: los siete jobs en DAG más `compile-latex` y `verify-checksums`; `build-images` depende de `lint`, las tres suites y la integración, y publica siete imágenes en GHCR etiquetadas por commit; la integración levanta el stack con sonda de salud y verifica el proveedor Pact y Playwright contra servicios reales; la ejecución sobre `964fa6e` terminó con 15 jobs en verde. La SPA, recompilada desde cero, pasa `tsc` y ESLint sin advertencias y 75 pruebas con cobertura de líneas 87,6 % (1.583/1.807) y de ramas 89,09 % (237/266) sobre todo `src/`; tiene las cinco rutas protegidas, `i18n` es/en, tema claro y oscuro y JWT en `sessionStorage` con expiración. El móvil guarda el JWT cifrado, tiene caché Room y actualización por arrastre y captura foto y GPS reales; el APK está firmado desde un secreto en CI. Command, Chain of Responsibility, Observer de escalado, Factory Method y Strategy de SLA y correlación son código real con 86 pruebas. Las cuatro métricas nuevas y tres `crdb_*`, registros JSON con `trace_id` y trazas hacia Tempo están instrumentados. Los tres manifiestos verifican (3/3, 41/41, 1/1) en CI; `generar_reporte_correl.py` regenera el análisis y la tabla del documento idénticos; `.gitattributes` deja cero archivos CRLF. El manuscrito compila a 55 páginas sin indefinidas, su PDF corresponde a la fuente y la tabla de aporte ya incluye a los cuatro integrantes.

2. Rúbrica del examen suspenso

Cómo se califica el examen suspenso

Sólo se califican los ítems de esta tabla: lo que está en Hecho no vuelve a puntuar ni se puede perder. El peso de cada ítem sale de sumar cuatro estimaciones de lo que cuesta llevarlo a Hecho con la calidad exigida: **profundidad** (P, 1–5: de aplicar una instrucción a rehacer análisis, diseño experimental o arquitectura con criterio propio), **complejidad** técnica (C, 1–5: de trámite mecánico a coordinar varios servicios, datos y herramientas), **amplitud** (A, 1–5: de un archivo a varios módulos, clientes, flujos y documentos a la vez) y **tiempo** de dedicación (T, 1–5: menos de 2, de 2 a 6, de 6 a 15, de 15 a 30 y más de 30 horas-persona), normalizando

a 100 dentro del equipo. Cada ítem recibe su peso multiplicado por el coeficiente del estado en que lo encuentre el viernes 18 de septiembre a las 23:55: **Hecho 1,00 · Por modificar 0,40 · Por culminar 0,20 · Por iniciar 0,00**. La nota del examen es la suma de puntos dividida entre diez. Un ítem sólo llega a Hecho si la comprobación descrita en “Cómo lo verifico” da el resultado exigido sobre el repositorio del docente; el mensaje de un commit, un archivo vacío o una cifra sin dato crudo que la respalde no cuentan. Las cláusulas de entrega de la primera página y la ausencia total de la aplicación web o de la móvil (factor 0,5) siguen vigentes sobre la nota.

§	Ítem que se califica	P	C	A	T	Pts	Peso %	Estado 13/09	Puntos
§1	URL canónica en CITATION.cff, portada y diapositiva; README con equipo, roles y avisos vigentes	1	1	2	1	5	2,3	Por modificar	0,92
§2	Raíz sin subproductos de compilación ni carpetas heredadas, mapa del Listado 3, rutas citadas existentes y variables del móvil	2	2	3	2	9	4,1	Por modificar	1,64
§3	Integración por PR con revisión de otro integrante, rama principal sin re-escrituras, CHANGELOG.md y etiqueta anotada de cierre	2	2	2	2	8	3,7	Por culminar	0,74
§4	Dominio sin Spring ni setters públicos, cuatro capas en todos los servicios con lógica, controladores sobre abstracciones y ArchUnit	3	3	4	3	13	5,9	Por modificar	2,36
§5	Strategy de rutas y Observer de SLA con uso real y pruebas; ADR-005 corregido y ADR-006 con comparación cuantitativa	3	3	3	3	12	5,5	Por culminar	1,10
§6	Vista de SLA por zona y por técnico sobre un endpoint real y rol de coordinador	2	3	3	3	11	5,0	Por culminar	1,00
§7	Compuerta de cobertura web ≥ 70 % en CI y artefacto dist versionado por commit	1	2	2	1	6	2,7	Por modificar	1,08
§8	Cierre en sitio que envía y persiste foto y GPS (con cola sin conexión) y listado de tickets asignados al técnico	3	4	4	4	15	6,8	Por culminar	1,36
§9	Pruebas de los tres ViewModels, E2E instrumentada login→lista→detalle y APK en Release inmutable de la etiqueta	2	3	3	3	11	5,0	Por culminar	1,00
§10	Contrato del consumidor móvil, contrato web ampliado a todas las operaciones y OpenAPI con rutas absolutas, esquemas y validación en CI	3	3	3	3	12	5,5	Por culminar	1,10
§11	Migraciones versionadas con Flyway, imágenes con versión fija y experimento de caída reproducible en la configuración E4	2	3	3	3	11	5,0	Por modificar	2,00
§12	Cobertura backend ≥ 70 % con compuerta en los cuatro servicios Java, Testcontainers en auth y report e informes como artefactos	3	3	4	4	14	6,4	Por culminar	1,28
§13	APK dependiente de lint y pruebas y agente OpenTelemetry con versión fija	1	2	2	1	6	2,7	Por modificar	1,08

§	Ítem que se califica	P	C	A	T	Pts	Peso %	Estado 13/09	Puntos
§14	Registros JSON y traceparent en el móvil, latencia medida en el cliente, histogramas y evidencia del tablero bajo carga	3	3	3	3	12	5,5	Por modificar	2,20
§15a	Protocolo versionado antes de medir y campaña CORREL con $r = 10$, variabilidad real, unidad de análisis correcta e IC exactos	4	3	3	4	14	6,4	Por culminar	1,28
§15b	Carga Locust conforme: 50 usuarios $\times$ 5 min y rampa 0→200 en 10 min, diez corridas por escenario	2	3	2	3	10	4,6	Por culminar	0,92
§16	Un juego canónico de resultados, manifiestos que cubren todos los datos, diccionario de datos y reproducción verificada en CI	3	2	3	2	10	4,6	Por culminar	0,92
§17	ISO/IEC 25010 completa: fiabilidad de 1 h, seguridad sobre todos los endpoints, mantenibilidad y compatibilidad con artefactos crudos	4	3	3	4	14	6,4	Por culminar	1,28
§18	Manuscrito con cifras canónicas, figuras de observabilidad y C4 nivel 3, estilo que imprima DOI/ISBN y versión congelada	2	2	4	3	11	5,0	Por culminar	1,00
§19	Amenazas de las campañas E4 con mitigación aplicada y reflexión ética $\geq \frac{1}{2}$ página con principios ACM	3	1	2	2	8	3,7	Por modificar	1,48
§20	Autoría transparente (.mailmap, declaración de commits por guion y reatribuidos), actas contemporáneas y aporte propio verificable	2	1	2	2	7	3,2	Por culminar	0,64
Total							100		

Lectura de la rúbrica al 13/09/2026, 19:00

La columna *Estado* es el que encontré en el commit de corte; la columna *Puntos* es peso por coeficiente. Nota provisional del examen suspenso: **2,64 / 10,00**. Si cierran todos los ítems en Hecho, la nota es 10,00.

3. Trabajo exigido

§1 Identificación del entregable y URL

Qué encontré: CITATION.cff (repository-code) y docs/diapositivas/identificacion_lms.tex siguen apuntando a carlospatroner-boop/PFC-SOPORTE-ISP; el manuscrito no contiene ninguna URL del repositorio; el README no tiene la URL, ni los integrantes, ni sus roles (sólo están en PLAN.md, un plan de la E3), conserva el aviso de la “fecha límite del viernes 28 de agosto” y dice que el documento tiene 51 páginas cuando tiene 55; CITATION.cff no declara versión. **Cómo debe quedar:** <https://github.com/gleiston-guerrero/PFC-SOPORTE-ISP> en README, CITATION.cff, portada del manuscrito y diapositiva; sección “Equipo” en el README con nombre completo, rol y usuario de GitHub; version y date-released coherentes con la etiqueta de cierre. **Cómo lo verifico:** `git grep -n carlospatroner-boop v4.0.0` no devuelve nada; `git grep -n gleiston-guerrero/PFC-SOPORTE-ISP v4.0.0 -- README.md CITATION.cff docs/latex/main.tex` devuelve las tres.

§2 Estructura — subproductos, carpetas heredadas y rutas que no existen

Qué encontré: versionan `main.bbl` y `main.blg`, 194 archivos HTML generados por JaCoCo y un `.exec` de 1,33 MB; `node_modules` de `notification-service` (1.575 archivos) sigue en el historial; conviven carpetas heredadas o ajenas sin explicación (`frontend/` de la E3, `PLAN.md`, `start-all.ps1`, `Evidencias de Modificaciones/` con capturas de “clic en editar”). Faltan ADR-001 y ADR-002, el C4 de nivel 3 en imagen, `iso25010.csv` y `boxplot_latencia.png` del Listado 3, y los siete servicios tienen nombres distintos sin tabla de correspondencia. `tolerancia_fallos.md` remite a `docs/evidencias/frames/` y una prueba E2E cita `rebalance_demo_tickets.sql`: ninguno existe. `.env.example` no tiene variables del móvil, cuya URL base `10.0.2.2:8000` está fija en el código. **Qué deben hacer:** retirar subproductos y añadir `*.bbl`, `*.blg` y `*.exec` a `.gitignore`; conservar de JaCoCo sólo `jacoco.xml` y `jacoco.csv`; mover lo heredado a `docs/entrega3/` o retirarlo; añadir al README la tabla “Listado 3 → ruta real”; crear los ADR ausentes o justificar su ausencia; corregir las rutas rotas; declarar la URL base del móvil como `buildConfigField` documentada en `.env.example`. **Cómo lo verifico:**

```
git ls-files | grep -E "\.(bbl|blg|aux|log|exec)$|jacoco-resources|node_modules" # vacío
grep -rn "frames/|rebalance_demo_tickets" --include=*.md --include=*.ts . # vacío
```

§3 Línea base — sin etiqueta de cierre, sin revisión y con historia reescrita

Qué encontré: las cuatro etiquetas existentes son ligeras y ninguna apunta a un commit de `main` (tres son copias de seguridad de reescrituras y `mobile-release` apunta a `f5300ed`, fuera de la rama). No hay `CHANGELOG`. De 13 PR anteriores al corte sólo el #1 tiene revisiones; cinco los abrió y fusionó la misma persona. Los 45 commits posteriores al último PR entraron directamente en `main`, y la rama se reescribió con *force-push* al menos tres veces (16 commits descartados el 09/09, siete reaplicados el 10/09 con otros autores y uno eliminado el 13/09). Los archivos del Release `mobile-release` se reemplazaron después del corte, así que no prueban nada sobre la entrega. **Cómo debe quedar:** rama `main` protegida (PR obligatorio, una aprobación, sin *force-push*); cada cambio del suspenso en un PR aprobado por un integrante distinto del autor; `CHANGELOG.md` con entradas fechadas de `v1.0.0` a `v4.0.0` y una del examen suspenso; etiquetas `backup-*` retiradas; etiqueta anotada de cierre sobre el último commit de `main`, creada **al final de todo**, con un Release inmutable ligado a ella. **Cómo lo verifico:**

```
git cat-file -t v4.0.0 && git merge-base --is-ancestor v4.0.0 origin/main
git for-each-ref refs/tags --format='%(%objecttype) %(%refname:short) %(%taggerdate:iso)'
git ls-remote --tags origin v4.0.0 # deben salir DOS líneas
```

y en la API, `/pulls?state=closed` con una revisión `APPROVED` de otro usuario en cada PR.

§4 Backend en capas — Spring dentro del dominio

Qué encontré: nueve archivos de `svc-principal/.../domain` llevan anotaciones de Spring (`@Component("c0")`, `@Component("c1")`, `@Value`, `@Order(1)`, en estrategias, cadena de escalado, fábrica y políticas de SLA), igual que `TelemetryStore`, lo que contradice el propio ADR-005 (“el dominio queda libre de anotaciones JPA/Spring”). Las entidades `Ticket`, `Incidencia`, `User`, `RefreshToken` y `TicketSummary` son anémicas con `@Setter` y `@AllArgsConstructor`, así que cualquier invariante se puede saltar. `telemetry-service` no tiene `application` ni `presentation`; `notification-service` y `ai-service` son planos. `TicketController` depende de tres manejadores concretos y no de `TicketCommandHandler<C,R>`. **Cómo debe quedar:** dominio sin `org.springframework`, `jakarta.persistence` ni setters públicos; beans contruidos en `infrastructure/config`; entidades con métodos de negocio (`asignar()`, `cambiarEstado()`, `escalar()`); cuatro capas en cada servicio con lógica, o un ADR que justifique los triviales; controladores sobre interfaces; prueba `ArchUnit` que lo haga cumplir. **Cómo lo verifico:**

```
grep -rln "import org.springframework\\lombok.Setter" services/*/src/main/java/ec/edu/uteq/soporte/*/domain #
↪ vacío
mvn -B test -Dtest=ArchitectureTest # en svc-principal, auth-service, report-service y telemetry-service
```

§5 Patrones GoF — faltan los dos que dependen del dominio del ISP

Qué encontré: Repository, Factory Method, Command, Chain of Responsibility, Observer de escalado y Strategy de SLA y de correlación son código real con pruebas. Pero la Tabla 1 exige un **Strategy de rutas** y no existe ninguna estrategia de rutas ni de orden de visitas en el backend ni en el móvil, y el **Observer de SLA** sólo cubre el escalado: ningún sujeto notifica a la consola ni al móvil el paso de un ticket a “en riesgo” o “vencido”. ADR-005 repite casi literalmente un párrafo del contexto, afirma que el dominio está libre de Spring (falso, §4) y cita 45 pruebas cuando hay 86. ADR-006 aporta cifras sólo de la opción elegida (APK de 11,06 MB y 20 dependencias) y reconoce que no midió ninguna alternativa. **Cómo debe quedar:** RutaStrategy con al menos dos implementaciones (p. ej. cercanía por GPS y prioridad por SLA) usadas por un caso de uso que expone la ruta del técnico y que el móvil consume, con pruebas; Observer de SLA que publique el cambio de estado de SLA hacia métrica, mensajería y consola, con pruebas; ADR-005 corregido y ADR-006 con una tabla de al menos dos criterios medidos o citados *para ambas opciones*. **Cómo lo verifico:** `grep -rn "RutaStrategy" services/svc-principal/src/main | wc -l ≥ 3`; `mvn -B test -Dtest='*Ruta*Test,*Sla*Observer*Test'` en verde; `grep -c "Antes de este refactor" docs/adr/0005-patrones-gof.md` da 1.

§6 Aplicación web — la vista de SLA es sólo un subtítulo

Qué encontré: la Tabla 1 pide una consola de operadores y coordinadores con vista de SLA por zona y por técnico. `es.json` tiene el subtítulo “Vista de SLA por zona y por técnico”, pero `ConsolePage.tsx` muestra tres indicadores globales, un filtro de zona y una insignia por fila; `ReportsPage.tsx` desglosa por estado, zona y categoría, sin técnico ni porcentaje de cumplimiento; `SummaryResponse` no expone agrupación por técnico. No existe el rol de coordinador u operador (roles: CLIENTE, TECNICO, ADMIN). **Cómo debe quedar:** una vista con tickets abiertos, vencidos, porcentaje de cumplimiento de SLA y tiempo medio de resolución por zona y por técnico, alimentada por un endpoint real del modelo de lectura (p. ej. `GET /api/v1/reports/sla?groupBy=zone|technician`), con i18n, estados de carga y error, pruebas Vitest y una prueba Playwright; rol de coordinador con acceso a ella. **Cómo lo verifico:** `grep -rn "groupBy\|byTechnician" services/report-service/src/main apps/web/src` NO VACÍO; `npx vitest run src/features/sla`; la respuesta del endpoint trae el porcentaje por técnico.

§7 Calidad web — la cobertura no es una compuerta

Qué encontré: la cobertura real es buena (véase el recuadro de lo hecho), pero el CI ejecuta `npm test` sin `--coverage` y `vitest.config.ts` no define umbrales: una caída por debajo del 70 % no rompería el pipeline. Ningún job publica `dist/` versionado por commit. **Qué deben hacer:** `coverage.thresholds` de 70 en las cuatro métricas; `npm run test:coverage` en el job con el informe como artefacto; paso de `npm run build` que suba `web-dist-{{ github.sha }}` con su suma; demostrar en un PR que la compuerta falla al subir temporalmente el umbral. **Cómo lo verifico:** `grep -n thresholds apps/web/vitest.config.ts`; artefactos `web-dist-<sha>` y `web-coverage` en la ejecución del commit final.

§8 Aplicación móvil — la evidencia se captura y se descarta

Qué encontré: inicio de sesión, listado con actualización por arrastre, detalle, caché Room y JWT cifrado funcionan, y la foto y el GPS se capturan de verdad. Pero `TicketDetailViewModel.closeOnSite()` sólo comprueba que existan y luego llama a `ticketRepository.closeOnSite(ticketId)` **sin foto ni coordenadas**; el propio `TicketRepository.kt` lo reconoce en un comentario (“hoy solo actualiza el estado a RESUELTO”), no hay endpoint multipart ni de evidencia en la app ni en el backend, y aun así el diálogo le dice al técnico que “se enviará la evidencia fotográfica y las coordenadas GPS”. Además el listado no muestra los tickets *asignados*: el backend filtra al técnico por zona, así que ve todos los de su zona. No hay

registros estructurados (Timber). **Cómo debe quedar:** cierre en sitio que sube la foto y la latitud y longitud (multipart u objeto almacenado), persiste la evidencia en una tabla creada por migración (§11) con ticket, técnico y marca de tiempo, la muestra en el detalle web y, sin conexión, la encola y sincroniza; listado móvil restringido a los tickets asignados al técnico autenticado. **Cómo lo verifico:**

```
grep -n "Multipart\\|latitude" apps/mobile/app/src/main/java/**/data/remote/TicketApi.kt
grep -rn "close-on-site\\|ticket_evidence" services/svc-principal/src
./gradlew :app:testDebugUnitTest --tests '*TicketDetailViewModelTest*'
```

más la evidencia versionada de un cierre real (petición y respuesta del gateway y fila en la tabla de evidencias).

§9 Calidad móvil — tres pruebas unitarias y ninguna E2E

Qué encontré: sólo LoginViewModelTest (3 pruebas); TicketListViewModel y TicketDetailViewModel, que contiene la lógica de cierre, no tienen pruebas. La única instrumentada valida campos vacíos del formulario y su comentario aclara que no depende del backend. El APK versionado se subió a mano; el Release depende de una etiqueta ligera fuera de main y sus archivos se sobrescriben en cada *push*. **Cómo debe quedar:** pruebas de los tres ViewModels (éxito, error, sin conexión y cierre con evidencia) con JUnit 5, coroutines-test y mockk; al menos una E2E instrumentada con servidor simulado que recorra inicio de sesión, lista y detalle; Release publicado sólo al empujar la etiqueta de cierre, sin sobrescritura, con SHA256SUMS.txt y enlace desde el README. **Cómo lo verifico:** `grep -c "@Test" apps/mobile/app/src/test/**/*ViewModelTest.kt` (tres archivos, 10 o más pruebas); `grep -rn MockWebServer apps/mobile/app/src/androidTest`; el APK del Release verifica con `sha256sum -c y apksigner verify`.

§10 Contratos — el cliente móvil no tiene contrato

Qué encontré: un único consumidor Pact, la web, con dos interacciones (ambas `GET /api/v1/tickets`: 401 sin testigo y 200 con ADMIN); no cubre detalle, creación, cambio de estado, asignación, inicio de sesión ni reportes. El móvil usa tres operaciones sin ningún contrato. La verificación del proveedor contra el stack real es correcta, pero sólo sobre esas dos interacciones. `docs/api/openapi.yaml` usa rutas relativas ambiguas (`/`, `/id`, `/summary`) bajo diez `servers` globales, deja sin esquema las respuestas 2xx de listar, cambiar estado, asignar y administrar usuarios, no documenta `GET /api/v1/tickets/incidencias`, y nada lo valida en CI. **Cómo debe quedar:** dos consumidores (soporte-web y soporte-mobile) con una interacción por cada operación que usa cada cliente, incluida la del cierre en sitio de §8, verificados por el proveedor real en CI; OpenAPI con rutas absolutas, esquemas en todas las respuestas y `npx @redocly/cli lint` en el job de lint. **Cómo lo verifico:** `ls pacts/` con dos archivos (web ≥ 5 y móvil ≥ 4 interacciones); registro del job de integración con “Verifying a pact between soporte-mobile and ticket-service” en verde; `redocly lint` sin errores.

§11 Persistencia distribuida — esquema sin historial de versiones

Qué encontré: tres nodos CockroachDB con localidades, healthchecks y `num_replicas = 3`. Pero no hay migraciones versionadas: el esquema se aplica con scripts `CREATE TABLE IF NOT EXISTS` desde el `command` de `db-init`, hay un `ALTER TABLE auth_db.users ADD COLUMN ... zone` incrustado en `docker-compose.yml`, y `docs/db/schema.sql` es una copia “de referencia” que puede divergir. La imagen es flotante (`latest-v23.2`), también en Testcontainers. La evidencia de tolerancia a fallos es de la E3 y cita capturas inexistentes. **Cómo debe quedar:** migraciones Flyway (`V1__init.sql`, `V2__users_zone.sql`, ...) en cada servicio con base de datos y `db-init` reducido a inicializar el clúster; versión fija de CockroachDB en compose y pruebas; script que detiene un nodo bajo carga y guarda un CSV con marcas de tiempo y su manifiesto, ejecutado sobre la configuración E4. **Cómo lo verifico:** `git ls-files "services/*/src/main/resources/db/migration/V*.sql"`

(≥ 2 por servicio con base); `grep -n "ALTER TABLE\\|latest" docker-compose.yml Vacío; sha256sum -c db-cluster/results/SHA256SUMS.txt.`

§12 Pirámide de pruebas — la cobertura del backend no se mide en CI

Qué encontré: 121 pruebas Java (86 en `svc-principal`, 15 `auth`, 11 `report`, 8 `telemetry`, 1 `gateway`), 75 `web`, 3 E2E Playwright, 3 unitarias y 2 instrumentadas móviles. Sólo `svc-principal` mide cobertura, en un perfil que el CI no activa y sin `check`: el 81,1 % de líneas procede de una ejecución local del 07/09 sobre un commit que no está en `main`, y sus ramas están en 57,1 %; `auth`, `report` y `telemetry` no tienen JaCoCo. Testcontainers sólo en tres pruebas de `svc-principal`. La carga no sigue la guía (§15b). El CI sólo sube el informe de Playwright. **Cómo debe quedar:** JaCoCo con `prepare-agent`, `report` y `check` al 70 % (instrucciones, ramas y líneas) ligados a `test` en los cuatro servicios Java, ejecutado en CI; pruebas de repositorio con Testcontainers en `auth` y `report`; informes de surefire, JaCoCo, Vitest, pruebas móviles y Playwright como artefactos. **Cómo lo verifico:**

```
grep -rn "<minimum>0.70" services/*/pom.xml | wc -l          # >= 4
for s in auth-service svc-principal report-service telemetry-service; do (cd services/$s && mvn -B test) ||
↪ echo FALLA $s; done
```

§13 Pipeline CI/CD — dos ajustes

Qué encontré: el pipeline es correcto (véase lo hecho) y baja sólo por lo que protege: no tiene compuertas de cobertura (§7, §12) ni verifica el contrato móvil (§10). Además `build-mobile-apk` no depende de lint y los Dockerfile descargan el agente OpenTelemetry desde `releases/latest`, lo que hace la imagen no reproducible. **Qué deben hacer:** `needs: [lint, test-mobile]` en el job del APK; URL del agente con versión fija y verificación de su suma. **Cómo lo verifico:** `grep -n "releases/latest" services/*/Dockerfile Vacío.`

§14 Observabilidad — el cliente móvil no emite nada

Qué encontré: el panel “latencia extremo a extremo desde el cliente móvil” consulta la latencia del gateway en el servidor; el móvil no emite métricas, trazas ni registros JSON con `trace_id`. `notification-service` y `ai-service` no exportan registros al Collector, que no tiene pipeline de registros. Los paneles de latencia promedian cuantiles de `summaries` entre instancias, lo cual no es válido estadísticamente. No hay ninguna captura del tablero ni de una traza, ni exportación de Prometheus, bajo carga. **Cómo debe quedar:** interceptor `OkHttp` que genere `traceparent` y mida la duración de cada petición, exportada por OTLP; Timber con árbol JSON que incluya `trace_id`; `http_request_duration_seconds` como histograma consultado con `histogram_quantile`; carpeta `docs/evidencias/observabilidad/` con capturas de los seis paneles y de una traza del móvil a la base de datos y respuestas `query_range` en JSON, todas con marcas de tiempo dentro de la campaña Locust de §15b.

§15 Campañas experimentales — diseño no cumplido

Qué encontré (CORREL): 60 filas = 3 escenarios $\times$ 3 modos $\times$ 5 réplicas, no 10; las cinco réplicas de cada una de las 12 celdas son **idénticas valor a valor** ($DE = 0$) y `c1` y `c2` coinciden fila a fila, porque el sistema es determinista y repetir no añade variabilidad; la población es de 20/60/20+20 abonados frente a los 100/500/100+100 del protocolo, y falta el escenario “Esc-4 en la misma zona” del ADR-0008. `protocolo-e4.md` entró **en el mismo commit que los datos** (8869188), con los resultados ya escritos, y el ADR-0008 no tiene hipótesis ni umbrales; aun así el documento dice “se rechaza H_0 ”. El intervalo “IC95 binomial [0,0]” es un Wald degenerado (con 0/30, Clopper-Pearson da [0; 0,116]); las “30 corridas” de Esc-4 son 15 corridas por dos zonas; el Mann-Whitney con $n = 20$ trata filas no independientes como réplicas; y la única marca de tiempo de la campaña

se borró del JSON el 10/09. **Qué encontré (Locust):** cinco corridas por escenario, con máximo de 30 usuarios durante 58–60 s (se exige 50 durante 5 minutos) y rampa de 0 a 60 en 119 s (se exige 0 a 200 en 10 minutos); tras descartes quedan $n = 3$; la columna de error etiquetada como 5xx incluye `ConnectionResetError`. **Cómo debe quedar:** protocolo con hipótesis, variables, umbrales, α , poblaciones del ADR y plan de análisis versionado *en un commit anterior al primer dato*; CORREL con $r = 10$ por celda, semilla distinta por réplica, orden aleatorio de modos, marcas de inicio y fin por réplica y árbol limpio registrado; unidad de análisis = corrida; intervalos exactos; Locust con `-u 50 -t 5m` y rampa 0→200 en 600 s, diez corridas por escenario. El cambio de prueba estadística de la E3 (hecho el 08/09) se declara como desviación del protocolo. **Cómo lo verifico:**

```
git log --format='%h %ad' --date=iso -- docs/experimentos/protocolo-e4.md | tail -1 # anterior a:
git log --format='%h %ad' --date=iso -- experimentos/resultados/correlacion_corridas.csv | tail -1
python -c "import csv,collections as c;r=list(csv.DictReader(open('experimentos/resultados/correlacion_corrid'
↪ as.csv')));print(c.Counter((x['escenario'],x['modo']) for x in r))"
```

y en cada `*_stats_history.csv`: máximo de `User Count` 50 con duración ≥ 300 s, o 200 con ≥ 600 s.

§16 Paquete de datos — manifiestos parciales que se regeneran solos

Qué encontré: los manifiestos verifican, pero dejan fuera `spark/results/**`, `db-cluster/results/*`, `docs/experimentos/resultados/*`, los datos de JaCoCo, la tabla generada del documento y los guiones de `experimentos/`; y `correr_campana.py` y `analizar_resultados.py` regeneran el manifiesto al final de cada ejecución (el de Locust se regeneró cuatro veces después de los datos), con lo que una suma nunca puede delatar un cambio. Hay dos juegos de resultados de Spark distintos ($T(1) = 352,75$ s frente a 235,22 s) sin declarar cuál es el canónico, y `docs/experimentos/resultados/` es una copia. No hay diccionario de datos. En Windows `generar_reporte_correl.py` escribe CRLF y la suma no coincide. `correr_campana.py` valida la cabecera después de recrear el contenedor y de `TRUNCATE`, y borra `verdad_campo.csv` sin respaldo. **Cómo debe quedar:** un árbol canónico único declarado en `resultados/README.md`; manifiestos que cubren todos los datos crudos, derivados y guiones, generados por un comando independiente que nunca se ejecuta al analizar; diccionario con columna, tipo, unidad, dominio y guion productor; `newline=""` en toda escritura; precondiciones validadas antes de cualquier efecto; `job reproduce` que regenera tablas y figuras y termina con `git diff --exit-code`. **Cómo lo verifico:** `grep -n SHA256SUMS experimentos/correr_campana.py resultados/locust/analizar_resultados.py` vacío; `sha256sum -c` en cada carpeta; el número de archivos de datos es igual al de líneas de los manifiestos.

§17 Evaluación ISO/IEC 25010 — tres características sin dato crudo

Qué encontré: eficiencia se reportó con 30 usuarios durante 60 s ($n = 3$), no con 50; fiabilidad exige una hora continua y **nunca se midió más de 119 s seguidos** (unos 15 minutos en total), y la causa atribuida a la licencia de CockroachDB no tiene registro versionado; seguridad es “30/30” con $n = 1$, sin artefacto crudo, sobre dos endpoints y sin inventario; mantenibilidad cita 542/668 y 2.529/3.518, cifras que no están en ningún archivo (JaCoCo dice 555/684 y 2.582/3.582), y la complejidad de `lizard` no está versionada; compatibilidad dice “9/9 Playwright” sin informe, WebKit no es Safari y el móvil no se ejecutó en API 26. En el manuscrito, los “19–34 %” de error son totales sin aplicar los descartes declarados y sin DE ni IC. El contraste con los umbrales (No / No / Parcial) sí es honesto. **Cómo debe quedar:** las cinco características con métrica objetiva, umbral, $n \geq 8$ réplicas válidas donde haya variabilidad, media, DE e IC95 % calculados por un guion versionado desde artefactos crudos versionados: fiabilidad con carga nominal de 60 minutos continuos; seguridad recorriendo todos los endpoints de OpenAPI sin testigo y con testigo inválido, más un informe ZAP; mantenibilidad con JaCoCo de los cuatro servicios y `lizard`; compatibilidad con informes de Playwright y ejecución en emuladores API 26 y 34; tabla del documento generada desde `iso25010.csv`. **Cómo lo verifico:** `python resultados/iso25010/calculador.py && git diff --exit-code`; duración del CSV de fiabilidad ≥ 3.600 s; `grep -rn "542/668\|2529/3518" docs` vacío.

§18 Manuscrito — cifras que no salen de la evidencia

Qué encontré: compila sin indefinidas, pero `pruebas_cicd.tex` declara “90/90 backend, 78/78 web, 5/5 móvil” (hay 121 pruebas Java y 75 web, y “5/5 móvil” contradice la amenaza que admite que las instrumentadas no se ejecutaron) y la discusión habla de “174 pruebas”; los porcentajes de Locust ignoran los descartes; mantenibilidad usa cifras inexistentes (§17). Observabilidad no tiene ninguna figura; la arquitectura sólo muestra C4 de niveles 1 y 2; la tabla ISO no tiene DE ni IC; el resumen repite “Abstract / Resumen”. `ieeetr` no imprime DOI ni ISBN (el PDF no contiene ninguno), hay diez referencias sin identificador y el ISBN de `kim2021devops` no existe. No hay versión congelada. La tabla de aporte no declara los commits generados por guion. **Cómo debe quedar:** conteos de pruebas y tablas ISO generados desde los informes y resultados canónicos; figuras del tablero bajo carga, de una traza y del C4 de nivel 3; `IEEEtran.bst` (0 `biblatex-ieee`) con DOI/ISBN visibles, 12 o más verificados e ISBN corregidos; URL canónica en portada; PDF congelado en `docs/entregas/E4/` con su suma y adjunto al Release de la etiqueta de cierre, compilado *después* de la última regeneración de resultados. **Cómo lo verifico:**

```
cd docs/latex && latexmk -pdf -interaction=nonstopmode main.tex && grep -c "Rerun\|undefined" main.log # 0
pdftotext main.pdf - | grep -ci "doi" # >= 12
pdftotext main.pdf - | grep -n "90/90\|78/78\|174 pruebas\|542/668" # vacío
```

§19 Amenazas a la validez y reflexión ética

Qué encontré: hay tres listas de amenazas (E3, ISO y proyecto); en la de E3 sólo una tiene mitigación aplicada, en la ISO ninguna, y en la del proyecto las dos externas no tienen mitigación, la interna 1 (“un solo desarrollador por capa”) contradice la tabla de aporte y la 3 se apoya en un archivo que no existe en ningún commit. Ninguna trata las amenazas reales de las campañas E4: réplicas deterministas, pseudorreplicación, carga distinta de la exigida y protocolo redactado después de medir. La reflexión ética tiene unas 221 palabras, cita principios genéricos sin numerar y afirma que el sistema “procesa datos personales reales”, en contradicción con los datos sintéticos declarados. **Cómo debe quedar:** una única sección con ≥ 3 internas y ≥ 2 externas que incluya las de las campañas E4, cada una con su mitigación aplicada y el artefacto que la respalda; reflexión de al menos media página con principios ACM numerados (1.2, 1.3, 1.6, 2.5) aplicados a la foto y ubicación del técnico, a los datos de abonados, al mínimo privilegio en JWT y a la honestidad sobre autoría y uso de IA.

§20 Trabajo en equipo y autoría — el historial no permite atribuir el aporte

Qué encontré: el commit `10703f1` añadió `auto_commits_timed.py`, que cambia `user.name` y `user.email` por los de los cuatro integrantes y crea 16 commits con pausas de 15–20 minutos; esos commits coinciden con la serie del 26/07, cuatro de ellos a nombre de Álvarez Párraga. El 21/08 se añadió `ejecutar_commits.py` a `.gitignore`. En la reescritura del 09–10/09, commits de Carpio Mendoza reaparecen con otro autor y *patch-id* idéntico (`df72093` → `286f9b3`, a nombre de Álvarez Párraga; `0f22773` → `1f6f016`). Las actas son reconstruidas el 04/09 desde un chat, registran asistencia completa por política y terminan el 01/09. Con su cuenta propia (DeJere), Álvarez Párraga tiene hasta el corte seis commits menores: 124 líneas de documentación, 35 de `.gitattributes`, una recompilación del PDF y un commit vacío. **Cómo debe quedar:** `.mailmap` que unifique las identidades de cada persona; nota de transparencia en actas y manuscrito sobre los commits generados por guion y los reatribuidos, con la tabla de aporte recalculada según ese método; actas levantadas el día de cada reunión del periodo del suspenso; y, para el estudiante que rinde el examen, un bloque sustantivo de los ítems de esta guía (sugeridos: §8, §9 y §12) hecho desde su propia cuenta, en PR revisados por otro integrante, que pueda explicar y defender. Sólo ese trabajo acredita su aporte. **Cómo lo verifico:**

```
git shortlog -sne --no-merges 964fa6e..v4.0.0 # con .mailmap
git log --author=DeJere --numstat 964fa6e..v4.0.0 -- apps/mobile services
```

```
grep -n "auto_commits_timed\|reatribu" docs/actas_reunion.md docs/latex/secciones/*.tex
```

y en la API, PR de DeJere aprobados por otro usuario y PR de otros revisados por DeJere.

4. Orden de ejecución

§3 protección de main y §20 .mailmap y flujo por PR (antes de tocar nada) → §4 dominio sin Spring → §11 migraciones → §8 cierre con evidencia y §5 rutas y SLA → §6 vista de SLA → §10 contratos y OpenAPI → §9 y §7 → §12 y §13 compuertas → §15 protocolo, código congelado y campañas (capturando §14 durante la carga) → §16 paquete de datos → §17 ISO/IEC 25010 → §18 y §19 con las cifras definitivas → §1 y §2 → CHANGELOG, manifiestos finales y, lo último, la etiqueta anotada con su Release.

5. Lista de verificación antes de entregar

```
git ls-remote https://github.com/gleiston-guerrero/PFC-SOPORTE-ISP HEAD
git cat-file -t v4.0.0 && git merge-base --is-ancestor v4.0.0 origin/main && test -f CHANGELOG.md
grep -rln "import org.springframework" services/*/src/main/java/ec/edu/uteq/soporte/*/domain
grep -n "Multipart\|latitude" apps/mobile/app/src/main/java/ec/edu/uteq/soporte/mobile/data/remote/TicketApi.kt
ls pacts/
for d in $(git ls-files | grep SHA256SUMS.txt | xargs -n1 dirname); do (cd $d && sha256sum -c --quiet
↪ SHA256SUMS.txt) || echo FALLA $d; done
cd docs/latex && latexmk -pdf main.tex && pdftotext main.pdf - | grep -ci doi
```

Documento de uso académico. Aplicaciones Distribuidas (ISR-701), UTEQ, 2026–2027 PPA. Verificación ejecutada sobre clon completo del repositorio declarado, con el estado al domingo 13 de septiembre de 2026 a las 19:00, el 16 de septiembre de 2026. Ing. Gleiston Cicerón Guerrero Ulloa, PhD.

Guía de cierre y rúbrica del examen suspenso

Equipo AGLS — TiendaTech, comercio electrónico distribuido con asistente de armado

Identificación	
Asignatura	Aplicaciones Distribuidas (ISR-701) — 2026–2027 PPA
Paralelo	7.º “A” Software
Repositorio evaluado	JoseLozanoMorales/TiendaTech
Cabecera evaluada	Rama main, commit 822399a — 341 commits — último commit anterior al domingo 13/09/2026, 19:00; verificación sobre clon completo, 16 de septiembre de 2026
Integrantes	Aucatoma Celorio Jhinson Stalyn · Lozano Morales José Alejandro · Gaibor Rodríguez Jeremy Ruperto · Sánchez Pilaloe Andy Paul
Rinden el examen suspenso	Aucatoma Celorio Jhinson Stalyn
Complejidad del proyecto	56 %
Corte del examen suspenso	Viernes 18 de septiembre de 2026, 23:55 (Ecuador)
Docente	Ing. Gleiston Cicerón Guerrero Ulloa, PhD.

Cláusulas de entrega del examen suspenso (anulan la calificación si no se cumplen)

1. Repositorio transferido y aceptado. El repositorio debe estar transferido a la cuenta del docente (gleiston-guerrero) y la transferencia **aceptada** antes del corte. Si al viernes 18/09/2026 a las 23:55 el repositorio no figura bajo github.com/gleiston-guerrero/, el examen se toma como **no entregado** y su nota es cero, cualquiera que sea el trabajo realizado. Estado al 13/09: **NO cumple**: el repositorio sigue bajo JoseLozanoMorales/TiendaTech (verificado también el 16/09). Es el único de los cuatro equipos que no lo ha transferido; sin transferencia aceptada no hay examen que calificar. Todo lo que se califica se revisa en esa URL y en su rama principal; lo que quede en ramas sin fusionar o en cuentas personales no existe para esta evaluación.

2. Carátula en una sola página subida al SGA. Cada estudiante que rinde el examen sube al SGA un único PDF de **una sola página**, sin capturas de cuestionarios, calificaciones ni contenido adicional, con: universidad, facultad y carrera; asignatura, paralelo y periodo; “Examen suspenso — Proyecto Fin de Curso”; nombre completo del estudiante que la sube; equipo y nombre del sistema; integrantes del equipo; docente; URL canónica del repositorio (<https://github.com/gleiston-guerrero/TiendaTech>) escrita completa en una línea y como enlace activo; rama, etiqueta anotada de cierre y hash corto del commit final; fecha de entrega. Una carátula de más de una página, con una URL distinta de la canónica o que no se suba al SGA equivale a examen no entregado para ese estudiante.

1. Evaluación de los entregables

#	Entregable / indicador	Estado propio	Estado efectivo	%	Acción
1	Identificación del entregable y URL	Por modificar	Por modificar	80 %	Véase §1
2	Estructura del repositorio y archivos de raíz	Por modificar	Por modificar	65 %	Véase §2
3	Línea base congelada e integración	Por culminar	Por culminar	40 %	Véase §3
4	Backend en capas y SOLID	Por modificar	Por modificar	80 %	Véase §4
5	Patrones GoF y ADR	Por culminar	Por culminar	35 %	Véase §5
6	Aplicación web funcional	Por culminar	Por culminar	55 %	Véase §6
7	Calidad de la aplicación web	Por modificar	Por culminar	55 %	Véase §7
8	Aplicación móvil funcional	Por modificar	Por modificar	60 %	Véase §8
9	Calidad de la aplicación móvil	Por modificar	Por modificar	65 %	Véase §9
10	Integración y contratos	Por culminar	Por culminar	30 %	Véase §10

#	Entregable / indicador	Estado propio	Estado efectivo	%	Acción
11	Persistencia distribuida	Por modificar	Por modificar	85 %	Véase §11
12	Pirámide de pruebas	Por culminar	Por culminar	45 %	Véase §12
13	Pipeline CI/CD	Por modificar	Por culminar	50 %	Véase §13
14	Observabilidad	Por modificar	Por modificar	55 %	Véase §14
15	Protocolo y campaña experimental	Por culminar	Por culminar	45 %	Véase §15
16	Paquete de datos y reproducibilidad	Por modificar	Por culminar	45 %	Véase §16
17	Evaluación ISO/IEC 25010	Por culminar	Por culminar	35 %	Véase §17
18	Manuscrito, bibliografía y trazabilidad E1–E4	Por modificar	Por culminar	45 %	Véase §18
19	Amenazas a la validez y reflexión ética	Hecho	Por modificar	85 %	Véase §19
20	Trabajo en equipo y autoría	Por modificar	Por modificar	55 %	Véase §20

Cómo leer las dos columnas de estado

El *estado propio* describe el artefacto en sí. El *estado efectivo* aplica la regla de dependencia: **ningún artefacto puede estar más completo que aquello de lo que se construye**. El backend en capas sostiene a los dos clientes; los clientes y el backend sostienen los contratos y la pirámide de pruebas; las pruebas sostienen el pipeline; el sistema instrumentado sostiene la campaña experimental; la campaña sostiene el paquete de datos; el paquete sostiene la evaluación ISO/IEC 25010; y el manuscrito, las amenazas a la validez y la línea base congelan y reportan todo lo anterior. Las actas, la declaración de uso de IA y los archivos de raíz no propagan: se declaran. Lo que se califica es el estado efectivo.

Lo que está hecho y no se recalifica

El manuscrito compila desde copia limpia a 59 páginas sin errores ni cajas desbordadas, su texto coincide con el PDF versionado, tiene 23 referencias (20 con DOI) y un validador de citas que corre en CI antes de compilar. Las amenazas a la validez (14, en cuatro tipos) y la reflexión ética (cinco párrafos aplicados a stock, pagos, credenciales, bibliografía e IA) están escritas con cuidado. El pipeline estaba en verde sobre 822399a (CI con 11 jobs, puerta de calidad con 15, integridad de datos, publicación multiarquitectura en GHCR por commit y despliegue con sonda). El APK de release está firmado por CI con certificado propio verificado con `apksigner`, y el JWT móvil se cifra con AES-GCM y Android Keystore. El manifiesto SHA-256 cubre los 60 CSV/TSV, verifica y se comprueba en CI con pruebas de alteración; las doce salidas estadísticas de la campaña se regeneran byte a byte; y el recálculo desde el crudo reproduce 120 filas, 30.275 checkouts confirmados, 43.168 órdenes persistidas y 13.210 inconsistentes, igual que el documento. Dominio Java sin anotaciones de framework en los seis servicios con 34 puertos y adaptadores; clúster CockroachDB de tres nodos con reintegración medida (3,62 s); complejidad ciclomática máxima de 9 con compuerta PMD; ESLint sin advertencias y TypeScript estricto; ramas integradas en `main`.

2. Rúbrica del examen suspenso

Cómo se califica el examen suspenso

Sólo se califican los ítems de esta tabla: lo que está en Hecho no vuelve a puntuar ni se puede perder. El peso de cada ítem sale de sumar cuatro estimaciones de lo que cuesta llevarlo a Hecho con la calidad exigida: **profundidad** (P, 1–5: de aplicar una instrucción a rehacer análisis, diseño experimental o arquitectura con criterio propio), **complejidad** técnica (C, 1–5: de trámite mecánico a coordinar varios servicios, datos y herramientas), **amplitud** (A, 1–5: de un archivo a varios módulos, clientes, flujos y documentos a la vez) y **tiempo** de dedicación (T,

1–5: menos de 2, de 2 a 6, de 6 a 15, de 15 a 30 y más de 30 horas-persona), normalizando a 100 dentro del equipo. Cada ítem recibe su peso multiplicado por el coeficiente del estado en que lo encuentre el viernes 18 de septiembre a las 23:55: **Hecho 1,00 · Por modificar 0,40 · Por culminar 0,20 · Por iniciar 0,00**. La nota del examen es la suma de puntos dividida entre diez. Un ítem sólo llega a Hecho si la comprobación descrita en “Cómo lo verifico” da el resultado exigido sobre el repositorio del docente; el mensaje de un commit, un archivo vacío o una cifra sin dato crudo que la respalde no cuentan. Las cláusulas de entrega de la primera página y la ausencia total de la aplicación web o de la móvil (factor 0,5) siguen vigentes sobre la nota.

§	Ítem que se califica	P	C	A	T	Pts	Peso %	Estado 13/09	Puntos
§1	Repositorio transferido, URL canónica en README, CITATION.cff, macros y portada; portada y roles actualizados	1	1	2	1	5	1,9	Por modificar	0,76
§2a	Sin configuración de IDE, binarios pesados, bases SQLite huérfanas ni <i>journals</i> versionados	1	2	3	2	8	3,1	Por modificar	1,24
§2b	Credencial de PayPal del historial revocada y documentada y detector de secretos en CI	2	1	1	1	5	1,9	Por iniciar	0,00
§2c	README sin afirmaciones falsas, mapa del Listado 3 y .gitattributes general	2	1	2	2	7	2,7	Por modificar	1,08
§3	Trabajo por PR aprobados por otro integrante, CHANGELOG.md y etiqueta anotada de cierre	1	1	2	2	6	2,3	Por culminar	0,46
§4	Capa de aplicación sin SQL, transacciones de framework ni excepciones HTTP; presentación sin infraestructura; ArchUnit	3	3	3	3	12	4,6	Por modificar	1,84
§5a	Strategy de pagos, Observer de pedidos, Decorator de promociones y Factory Method en el flujo real de checkout, con pruebas	3	3	3	4	13	5,0	Por iniciar	0,00
§5b	ADR-006 de elección móvil con alternativas nombradas y dos criterios medidos	2	1	1	1	5	1,9	Por culminar	0,38
§6	SPA: /main, /settings, /about, guardas JWT, i18n es/en, token en memoria, checkout funcional, panel de vendedor, métricas de administrador y cabecera de identidad segura	3	3	4	4	14	5,4	Por culminar	1,08
§7	Cobertura web ≥ 70 % sobre todo <code>src/</code> con umbrales que fallen, artefacto dist por commit y contenedor no root	2	3	3	3	11	4,2	Por culminar	0,84
§8	Notificaciones push por cambio real de estado del pedido, escáner resuelto contra el backend y actualización por arrastre	3	4	3	4	14	5,4	Por culminar	1,08
§9	Pruebas de los ViewModels, E2E instrumentada, APK del repositorio igual al del CI y <code>versionCode</code> incremental	3	3	3	3	12	4,6	Por modificar	1,84

§	Ítem que se califica	P	C	A	T	Pts	Peso %	Estado 13/09	Puntos
§10	Contratos sobre los clientes reales (web y Kotlin) verificados por los proveedores con estados, en un job que pueda fallar	3	3	3	3	12	4,6	Por culminar	0,92
§11	Migraciones completas aplicadas por herramienta y prueba de caída con escrituras concurrentes repetida	2	2	2	2	8	3,1	Por modificar	1,24
§12a	Cobertura backend ≥ 70 % por servicio sin filtros de alcance, con compuerta, incluido el gateway	3	3	5	5	16	6,2	Por culminar	1,24
§12b	Testcontainers en tres servicios, E2E de compra contra el stack real y Locust nominal y rampa con checkout	2	3	3	3	11	4,2	Por culminar	0,84
§13	DAG único que condiciona la publicación a todas las pruebas e integración con compose y sonda de salud	3	3	2	3	11	4,2	Por culminar	0,84
§14	Métricas con los nombres exigidos, scrape de CockroachDB y contenedores, Collector, seis paneles y registros del móvil	3	3	4	3	13	5,0	Por modificar	2,00
§15a	Protocolo con hipótesis, umbrales, potencia, $r = 10$ y orden aleatorizado versionado antes de medir	4	2	2	2	10	3,8	Por culminar	0,76
§15b	Campaña 2PC/Saga sin saturación, con oráculo prospectivo, una sola población y recursos medidos	4	4	4	5	17	6,7	Por culminar	1,34
§16	Paquete canónico único, diccionario de datos, manifiesto de todos los datos y volcado que haga regenerable el oráculo	3	3	4	3	13	5,0	Por culminar	1,00
§17	ISO/IEC 25010 con las cinco características de la Tabla 2 (incluida compatibilidad) y réplicas reales	3	3	3	4	13	5,0	Por culminar	1,00
§18	Manuscrito con la estructura de la Tabla 3, tablas generadas desde CSV canónicos, portada, compilación limpia documentada y hash vigente	3	2	4	4	13	5,0	Por culminar	1,00
§19	Amenazas actualizadas a la nueva campaña con mitigación explícita y ACM Code of Ethics 2018 aplicado	2	1	1	1	5	1,9	Por modificar	0,76
§20	Actas por sesión coherentes con el historial, PR aprobados por otro integrante y autoría por ítem	1	1	2	2	6	2,3	Por modificar	0,92
Total							100		

Lectura de la rúbrica al 13/09/2026, 19:00

La columna *Estado* es el que encontré en el commit de corte; la columna *Puntos* es peso por coeficiente. Nota provisional del examen suspenso: **2,45 / 10,00**, condicionada a la cláusula 1: si el repositorio no está transferido y aceptado al corte, la nota es cero. Si cierran todos los ítems en Hecho, la nota es 10,00.

3. Trabajo exigido

§1 Identificación del entregable y URL — repositorio sin transferir

Qué encontré: la URL `JoseLozanoMorales/TiendaTech` es coherente en README, CITATION.cff y la macro de enlaces del manuscrito, pero **el repositorio no está transferido** (cláusula 1). El commit de corte quitó la URL de la portada y dejó “Proyecto Fin de Curso — Paso 13”, “Corte documental: 1 de septiembre de 2026” y “Período académico 2026”. Gaibor Rodríguez figura como “Desarrollador” en el manuscrito y como “Líder de Desarrollo” en el README. CITATION.cff declara `version: "4.0.0"` con fecha 01/09 sin que existiera esa etiqueta. **Qué deben hacer:** transferir el repositorio a `gleiston-guerrero` y confirmar que la transferencia fue aceptada; `git remote set-url origin https://github.com/gleiston-guerrero/TiendaTech.git`; sustituir la URL en README, CITATION.cff, macros y enlaces del manuscrito; rehacer la portada con la URL, periodo 2026–2027, “Entrega 4 — examen suspenso” y fecha real; unificar roles; `date-released` igual a la fecha de la etiqueta de cierre. **Cómo lo verifico:**

```
git ls-remote https://github.com/gleiston-guerrero/TiendaTech HEAD
grep -rn "JoseLozanoMorales/TiendaTech" README.md CITATION.cff docs/entrega4/*.tex # vacio
pfdtotext -l 1 docs/entrega4/PFC4.pdf - | grep -E "github.com/gleiston-guerrero|2026.2027"
```

§2 Estructura — peso muerto, un secreto en el historial y un README que afirma lo que no hay

a) Higiene. Siguen versionados `.idea/TiendaTech.iml` (con una excepción expresa en `.gitignore`) y siete archivos de `Apps/mobile/.idea/`; un vídeo de 92 MB, un APK de depuración de 40 MB, el agente OpenTelemetry de 22 MB, un `compilacion.log` y **358 bases SQLite**, 240 de ellas en `resultados-rubrica-20260904/` con un `.db-journal` de una transacción sin cerrar; el paquete del repositorio pesa 230,55 MiB. **Deben:** retirar la configuración de IDE a cualquier profundidad (`**/.idea/`, `*.iml`), publicar vídeo y bases como archivos de un Release enlazados, descargar el agente en el Dockerfile verificando su suma y retirar el APK de depuración y el log.

b) Secreto. El commit `396b1b3` (26/07) versionó `paypal.clientId` y `paypal.clientSecret` de una aplicación *sandbox*; se retiraron 73 minutos después, pero siguen legibles en el historial. **Deben:** revocar y rotar esa credencial, dejar constancia fechada (sin el valor) en `docs/evidencias/` y añadir `gitLeaks 0 trufflehog` como job del CI.

c) README y raíz. El README marca como “Completo” los cinco patrones GoF que el propio ADR-005 admite que no existen (§5), dice en otra sección que no hay Pact, Playwright ni lint web, y afirma que la publicación “se condiciona” a contratos y E2E, lo cual es falso (§13). No hay ADR-006, `protocolo-e4.md`, `ops/grafana/pfc-dashboard.json`, `ops/otel-collector/config.yaml` Ni `release/apk/`, y nada mapea esas diferencias. `.gitattributes` no tiene regla general. **Deben:** reescribir las tablas de estado del README con estados verificables, añadir la tabla “Listado 3 → TiendaTech” y declarar `* text=auto eol=lf` con las excepciones binarias. **Cómo lo verifico:**

```
git ls-files | grep -iE "(~/)\.idea/|\.iml$|\.db-journal$|\.mp4$|debug\.apk$|\.jar$" | grep -v
↪ gradle-wrapper.jar # vacio
git ls-files --eol | awk '{print $1}' | sort | uniq -c
```

§3 Línea base — casi nada pasó por revisión

Qué encontré: ninguna etiqueta anotada (`pre-e4` y las 35 `mobile-*` creadas por CI son ligeras; `v4.0.0` es ligera y posterior al corte). No hay CHANGELOG. De 21 fusiones en la rama principal sólo 3 son PR, y **sólo el #79 tiene la aprobación de otro integrante**; el #84 se fusionó cuatro minutos después de abrirse. Desde el 01/09 entraron 82 commits directos a `main`. **Cómo debe quedar:** protección de `main` con una aprobación y verificaciones obligatorias; todo el trabajo del suspenso en PR abiertos por su autor y aprobados por otro integrante; `CHANGELOG.md` con entradas fechadas (E1–E4 y examen suspenso); la etiqueta ligera `v4.0.0` retirada y una etiqueta anotada de cierre (p. ej. `v4.1.0`) sobre el último commit de `main`, creada **al final de todo**. **Cómo lo verifico:**

```
git cat-file -t v4.1.0 && git rev-parse v4.1.0~{commit} origin/main
git ls-remote --tags origin v4.1.0 # deben salir DOS líneas
```

y en la API, cada PR del suspenso con una revisión APPROVED de un usuario distinto del autor.

§4 Backend en capas — aplicación que conoce SQL y HTTP

Qué encontré: el dominio está limpio y hay 34 puertos con adaptador, pero StockReservationService (aplicación de inventario) recibe JdbcTemplate y ejecuta SELECT ... FOR UPDATE; CrdbTransactionRetryExecutor USA TransactionTemplate; OtpService USA BCrypt aunque existe el puerto PasswordHasher; UsuarioService, OrdenCompraService, ProveedorService, OrdenService y FacturaService lanzan ResponseStatusException; y tres controladores importan infraestructura concreta (servidores TCP/gRPC, inyector de fallos experimental). **Cómo debe quedar:** aplicación que sólo depende de puertos y excepciones de dominio; SQL y transacciones en adaptadores; traducción a HTTP en un @ControllerAdvice; reglas ArchUnit en cada servicio. **Cómo lo verifiqué:** `grep -rn "JdbcTemplate\|ResponseStatusException\|TransactionTemplate\|BCrypt" services/*/src/main/java/**/application` vacío; la prueba de arquitectura en verde en CI.

§5 Patrones GoF — cuatro de los cinco exigidos no existen

Qué encontré: la Tabla 1 exige Repository, Factory Method, Strategy de métodos de pago, Observer de eventos de pedido y Decorator de promociones. Sólo Repository es real. MetodoPago y TipoMetodoPago son POJO y OrdenService sólo pasa un entero metodopagoId; no hay ningún publicador ni oyente de eventos de pedido; no existe ninguna clase de promoción o descuento; y el propio ADR-005 reconoce que las “fábricas” no son Factory Method. Los patrones reales del proyecto (Strategy, Adapter, Chain of Responsibility y Template Method en armado-ia; un Decorator de cabeceras en el gateway) no están en el dominio de venta. No existe ADR-006; la decisión móvil está en 002-mobile-platform.md con criterios triviales (“1 de 1”) y sin alternativas nombradas. **Cómo debe quedar:** en el flujo real de checkout, PaymentStrategy con dos o más implementaciones elegidas según TipoMetodoPago; publicador de eventos de pedido con dos o más suscriptores reales (facturación y la notificación push de §8); Decorator de promociones apilable aplicado al total del carrito; Factory Method con creador abstracto y concretos; cada uno con una prueba que demuestre la variación, y ADR-005 con clase y archivo de cada patrón. ADR-006-eleccion-movil.md en formato Nygard con Flutter, React Native o KMP como alternativas y al menos dos criterios medidos con cifra y fuente. **Cómo lo verifiqué:**

```
grep -rlnE "interface \w*(Payment|Pago)\w*Strategy|interface \w*Observer|\w*Decorator
↪ (implements|extends)|abstract class \w*Creator" services/*/src/main/java
grep -cE "~## (Estado|Contexto|Decisi|Consecuencias)" docs/adr/ADR-006* # 4
```

y compruebo que OrdenService invoca la estrategia y que retirar una implementación hace fallar su prueba.

§6 Aplicación web — el checkout no compra

Qué encontré: doce rutas de dominio, pero faltan /main, /settings y /about; sólo /admin tiene guarda, y decide por el rol guardado en sessionStorage sin validar el JWT ni su expiración; /carrito, /trabajador y /factura no están protegidas. No hay internacionalización. CheckoutView.tsx contiene un botón disabled con el texto “Pago pendiente de integración” y nunca llama a /api/ordenes/checkout (ya estaba como pendiente en el acta del 04/09). El panel de vendedor muestra dos contadores y dos enlaces; el panel de administrador cuenta productos, categorías y usuarios, sin métricas de venta. Además el cliente envía X-Usuario-Id, el gateway no la sobrescribe y OrdenCompraController confía en ella: un usuario puede actuar en nombre de otro. **Cómo debe quedar:** las cinco rutas de la guía más las del dominio, con guarda que valide exp en todas las privadas; i18n es/en con selector persistente; testigo de acceso en memoria

con renovación por la cookie `httpOnly` ya existente; checkout funcional hasta la factura usando la Strategy de §5; panel de vendedor con sus productos, stock y pedidos por estado; panel de administrador con ventas por día, ticket medio y productos más vendidos desde un endpoint agregado; X-Usuario-Id eliminada o sobrescrita en el gateway con el sub del testigo. **Cómo lo verifico:** `grep -nE 'path="/(main|settings|about)'" src/App.tsx; ls src/locales/{es,en}.json; grep -n disabled src/views/CheckoutView.tsx` vacío; una petición con X-Usuario-Id de otro usuario devuelve 403; la E2E de compra de §12 en verde.

§7 Calidad web — cobertura de alcance elegido y sin umbral

Qué encontré: TypeScript estricto y ESLint limpio. Pero `vitest.config.ts` enumera doce vistas en `coverage.include`, excluye `App.tsx`, `api.ts`, `cart.ts` y `session.ts`, y **no tiene** `thresholds`: la compuerta no puede fallar. En mi ejecución (62 pruebas en verde) las sentencias cubiertas son 56,08 % con el alcance configurado y 56,22 % sobre todo `src`; el 79 % de líneas está inflado porque los componentes están escritos en muy pocas líneas (`AdminView.tsx`: 48 KB en 138 líneas). La evidencia enlazada en el README es del 11/09 (3 vistas, 30,78 % de sentencias). El commit que dice “generar dinámicamente el alcance de cobertura” sólo cambia el texto del resumen del CI. No hay artefacto `dist` por commit; el Dockerfile corre como `root` y sin `HEALTHCHECK`. **Cómo debe quedar:** `include: ['src/**/*.{ts,tsx}']` con exclusiones justificadas, `thresholds` de 70 en las cuatro métricas y el CI en rojo si no se alcanzan; componentes formateados; evidencia regenerada desde la última ejecución; artefacto `webapp-dist-<sha>` con su suma; contenedor con `USER` no `root` y `HEALTHCHECK`. **Cómo lo verifico:**

```
cd Apps/web/frontend/webapp && npm ci && npx vitest run --coverage; echo "exit=$?"
node -e "console.log(require('./coverage/coverage-summary.json').total.statements.pct)" # >= 70
```

§8 Aplicación móvil — ni push ni escáner conectados al sistema

Qué encontré: inicio de sesión, catálogo, carrito y checkout contra el backend y JWT cifrado funcionan. Pero **las notificaciones push de estado del pedido no existen**: no hay Firebase, el publicador sólo se dispara con un botón de demostración y la propia pantalla dice “No existe configuración Firebase, registro de dispositivo ni eventos de estado en el backend”. La cámara y ML Kit son reales, pero el código leído se busca en **una tabla fija de cinco EAN** de demostración, no en el backend. No hay actualización por arrastre y el modo sin conexión sólo cubre productos y categorías. El README llama “segunda capacidad” a la caché Room. **Cómo debe quedar:** notificación push (FCM, o como mínimo `WorkManager` documentado) disparada por cambios reales de estado del pedido desde el Observer de §5; columna `codigo_barras` con migración, endpoint de búsqueda y repositorio remoto con caché; `PullToRefreshBox` en catálogo y pedidos; evidencia en `release/screenshots/` de un pedido que cambia de estado y la notificación recibida sin pulsar ningún botón de demostración. **Cómo lo verifico:** `grep -rn "firebase-messaging\\|FirebaseMessagingService\\|WorkManager" Apps/mobile/app; grep -rn "codigo_barras" services Apps/mobile; grep -rn PullToRefresh Apps/mobile/app/src/main.`

§9 Calidad móvil

Qué encontré: 58 pruebas unitarias, pero **sólo 1 de 11 ViewModels** (el del escáner) tiene pruebas; las cuatro instrumentadas son la plantilla, la base de datos y la notificación, y ninguna recorre la interfaz. La firma de release quedó resuelta, pero `release/tiendatech-release.apk` es una compilación local que no coincide con el binario del CI (35.849.002 frente a 35.849.007 bytes), el job de calidad móvil sólo verifica el APK de depuración, y `versionCode` es 1 con R8 desactivado. **Cómo debe quedar:** pruebas de al menos Auth, Catalog, Cart, Orders y Account con `kotlinx-coroutines-test`; una E2E instrumentada con `createAndroidComposeRule<MainActivity>()` y servidor simulado (inicio de sesión, catálogo, carrito) ejecutada en CI con emulador; el APK del repositorio igual al del Release de CI (o sólo el enlace); `versionCode` desde `GITHUB_RUN_NUMBER`. **Cómo lo**

verifico: `grep -rl ViewModel Apps/mobile/app/src/test | wc -l ≥ 6; grep -rn createAndroidComposeRule Apps/mobile/app/src/androidTest; sha256sum` del APK igual al publicado por CI.

§10 Contratos — no verifican a ningún proveedor

Qué encontré: dos consumidores en Node con una interacción cada uno (`GET /api/productos` y `POST /api/login`) que llaman al simulador con `fetch`: no usan el cliente real de la web ni el Retrofit de Kotlin (el “consumidor móvil” es JavaScript). No hay verificación de proveedor en ningún servicio y el job sólo genera y sube JSON, así que no puede fallar. Y los contratos no coinciden con los proveedores: `ApiResponseAdvice` envuelve las respuestas en `{status,data,message,timestamp}` y el contrato espera un arreglo; el móvil espera `ApiEnvelope<LoginResponse>` y el contrato `{user, access}`.

Cómo debe quedar: consumidores escritos sobre `src/services/api.ts` y sobre la interfaz Retrofit, con al menos inicio de sesión, catálogo y checkout; verificación con `@Provider`, `@PactFolder` y `@State` en usuarios, productos y pedidos; job `contract-verify` que falle si un proveedor cambia la forma de la respuesta. **Cómo lo verifico:** `grep -rln "@Provider\\|PactVerificationContext" services;` en una rama altero un campo del sobre de respuesta y el job se pone en rojo.

§11 Persistencia distribuida

Qué encontré: tres nodos con healthchecks, evidencia de fragmentación y una prueba de caída coherente con su commit. Pero `docs/db/migrations/` sólo tiene V004–V006 (faltan V001–V003), ninguna herramienta las aplica (el `crdb-init` carga `schema.sql`), y la prueba de caída es una sola ejecución de lectura, sin escrituras durante la caída. **Cómo debe quedar:** migraciones continuas desde V001 aplicadas por Flyway (por servicio o como servicio del compose); prueba de caída con inserciones concurrentes repetida tres veces que compruebe que confirmadas = persistidas, con CSV de punto decimal y sin BOM. **Cómo lo verifico:** `grep -rn flyway services/*/pom.xml docker-compose.yml;` CSV con tres filas y `perdidas=0`.

§12 Pirámide de pruebas — la compuerta del backend mide la capa que conviene

a) Cobertura del backend. Los seis `pom.xml` exigen 70 % de líneas sólo sobre `application/**` (usuarios, sólo sobre `application/service/**`); el gateway no tiene JaCoCo; y el 82,31 % que publica el documento sale de informes del 28/08 filtrados (inventario: 8 líneas medidas). **Deben:** quitar los `<includes>` restrictivos, excluir sólo clases de arranque y configuración justificadas, añadir JaCoCo al gateway y escribir pruebas de dominio, adaptadores JDBC y controladores (`@WebMvcTest`) hasta alcanzar 70 % por servicio con compuerta en CI. **b) Integración, E2E y carga.** Testcontainers sólo en pedidos; las tres pruebas Playwright simulan todas las APIs con `page.route` y no cubren carrito ni checkout; no hay Espresso; Locust sólo ejecuta cuatro GET públicos durante 60 s en una única corrida, sin el escenario de 5 minutos ni la rampa 0→200. **Deben:** Testcontainers en inventario, ventas y usuarios; E2E de compra sin `page.route` contra el compose; Locust con tareas autenticadas (inicio de sesión, carrito, checkout), `--users 50 --run-time 5m` y `LoadTestShape 0→200` en 600 s, con CSV, HTML y sumas versionados. **Cómo lo verifico:**

```
grep -n "<include>" services/*/pom.xml # vacío o solo exclusiones justificadas
grep -n "LoadTestShape\\|checkout" tests/load/locustfile.py
grep -c "page.route" tests/e2e-web/tests/compra.spec.js # 0
```

§13 Pipeline CI/CD — la publicación no espera a todas las pruebas

Qué encontré: existe un equivalente de cada job y todo estaba en verde, pero las compuertas no pueden fallar por lo que dicen medir (§7, §10, §12) y Codecov tiene `fail_ci_if_error: false`. El DAG está partido: `publish-images.yml` sólo espera a “CI-CD quality gate”, de modo que contratos, E2E, web, Android y manuscrito (workflow “CI”) no condicionan la publicación; `workflow_dispatch` publica sin compuerta; y la integración no levanta el compose ni sondea la salud del stack. **Cómo**

debe quedar: un único DAG en el que la publicación dependa de lint, pruebas backend, web y móvil y verificación de contratos; job `integration` que levante el compose con las imágenes del commit y haga `curl --fail --retry 30` a la salud del gateway y los servicios más un checkout de humo; publicación y despliegue sólo desde ese DAG. **Cómo lo verifico:** en una rama con una prueba web rota, la publicación no se ejecuta; `grep -n "curl --fail" .github/workflows/*.yaml` en el job de integración.

§14 Observabilidad — tablero incompleto y métricas con otros nombres

Qué encontré: registros JSON en los seis servicios, trazas con el agente OpenTelemetry hacia Jaeger y una traza de compra versionada. Pero el tablero tiene cinco paneles y le faltan disponibilidad Raft, recursos por contenedor y latencia desde el móvil; las métricas se llaman `request_count`, `request_duration` y `active_connections`; `app_business_events_total` y las `crdb_*` sólo existen en pedidos; Prometheus no recoge CockroachDB, el gateway ni cAdvisor; no hay OpenTelemetry Collector con su `config.yaml`; el móvil no emite registros JSON. **Cómo debe quedar:** `http_requests_total`, `http_request_duration_seconds`, `app_business_events_total` y `app_active_sessions` en los seis servicios; *scrape* de `/_status/vars` de los tres nodos, del gateway y de cAdvisor; Collector en el compose; `ops/grafana/pfc-dashboard.json` con los seis paneles, incluida la latencia medida en el cliente móvil; Timber con árbol JSON y `traceparent`; captura de los seis paneles con datos durante la carga de §12.

§15 Campaña 2PC/Saga — un diseño que no podía dar resultado

a) Diseño. Hay 5 réplicas por condición ($24 \times 5 = 120$), no 10, y la tabla `tab:protocolo` del manuscrito atribuye las cinco réplicas a la guía, lo cual es falso. El propio documento admite que la formulación “no se presenta como un protocolo preespecificado antes de la corrida”. Con $n_1 = n_2 = 5$, el p exacto mínimo de Mann–Whitney es $2/252 = 0,0079$, mayor que el umbral de Bonferroni $0,05/12 = 0,0042$: **ningún contraste podía ser significativo**. El orden de ejecución fue fijo. En E3 se cita `tiempos_crudos.csv` (1, 2 y 4 ejecutores, cinco repeticiones), no el juego que sí cumple ($N = 1, 2, 4, 8$ y $r = 10$). **b) Ejecución.** Recalculado desde el crudo: 47,26 % de solicitudes fallidas (68,9–76,6 % con $c = 400$); medianas de p95 de 30–37 s con $c = 50$ y de 86–92 s con $c \geq 200$, y en tres condiciones los cinco p95 valen exactamente 90.000 ms: **el sistema estaba saturado y el p95 mide el tiempo límite del cliente**, no la coordinación. La consistencia, que es la variable central, se reconstruyó 19 horas después desde una instantánea, porque los estados finales estaban en un búfer en memoria de 200 entradas; el invariante de compensación queda `no_verificado` en las 120 filas. Se mezclan poblaciones: 30.275 confirmaciones HTTP frente a 43.168 órdenes persistidas. La CPU de Locust vale 0,0 en 82 de 120 filas. La campaña anterior (3 checkouts, 74 % de error) sigue versionada junto a la canónica. **Cómo debe quedar:** `protocolo-e4.md` con H0/H1 por variable, δ , α , análisis de potencia, $r = 10$ con descarte de la primera y la última, orden aleatorizado con semilla y criterios de descarte, versionado *antes* del primer dato (si no hay tiempo para 240 corridas, reducir condiciones y declararlo ahí); nivel de carga por debajo de saturación elegido con la rampa (error < 5 % sin fallos); cada intento persistido con su clave de idempotencia, su estado HTTP y su estado final, verificado por el oráculo al final de cada corrida y antes del reinicio, con un control positivo que inyecte una violación conocida; una sola población para latencia e invariantes; CPU y memoria medidas; la campaña anterior movida a `historico/`. **Cómo lo verifico:**

```
git log --format='%h %cI' -- docs/experimentos/protocolo-e4.md | tail -1 # anterior al primer inicio_epoch
python -c "import csv,collections as c;r=list(csv.DictReader(open('<crudo
-> nuevo>')));print(set(c.Counter((x['fallo'],x['coord'],x['concurrency']) for x in r).values()))" # {10}
```

§16 Paquete de datos

Qué encontré: el manifiesto de los 60 CSV/TSV quedó cerrado, pero quedan fuera 358 .db, 54 .json (preflight, pilotos, rampa, resumen del oráculo, trazas) y los XML de PMD y JaCoCo; se regenera en el mismo commit que cambia los datos y el hash del crudo canónico no está fijado en el documento. Conviven cinco juegos de resultados (simulación SQLite duplicada, oficial-v4, resultados-rubrica sin referencias, la correctiva y dos juegos PySpark) y dos tablas de p para el mismo contraste (aproximado y exacto). Las columnas de inconsistencia dependen de una instantánea de un clúster que no se conserva, así que no son regenerables. No hay diccionario de datos. **Cómo debe quedar:** un directorio canónico por experimento con los históricos declarados; diccionario con nombre, tipo, unidad y origen de las columnas del crudo y del oráculo; manifiesto que cubra todo archivo de datos con su hash raíz citado en el manuscrito; volcado de las tablas usadas por el oráculo para que `reconstruct_real_oracle.py --desde-volcado` lo regenere; una sola tabla de p exactos. **Cómo lo verifico:** `python scripts/check_data_checksums.py` y el número de archivos de datos igual al de líneas del manifiesto; la regeneración del oráculo desde el volcado coincide con `cmp`.

§17 Evaluación ISO/IEC 25010 — sin compatibilidad y sin réplicas

Qué encontré: falta Compatibilidad (se sustituyó por Disponibilidad). Eficiencia ($p_{95} = 610$ ms) sale de una sola corrida de 60s con cuatro GET públicos y el límite de tasa del gateway elevado; la fiabilidad “0/2.112” sale de esa misma carga de 60s, y la hora de disponibilidad se midió después y sin carga (un GET por segundo). Mantenibilidad usa el 82,31 % filtrado (§12a) y reporta la complejidad máxima, no la media. Seguridad comprueba 8 familias con 401, sin ZAP ni auditoría de dependencias. Los IC95 son de Wilson sobre líneas o sondeos: **ninguna característica tiene réplicas**. **Cómo debe quedar:** las cinco características de la Tabla 2 con $r \geq 10$ (8 útiles), media, DE e IC95 % de t de Student sobre réplicas: fiabilidad en una hora bajo carga nominal con checkout; eficiencia como p_{95} por réplica de 5 minutos; seguridad como porcentaje de endpoints con JWT más informe ZAP *baseline* y `npm audit`; mantenibilidad con la cobertura completa de §12a y complejidad media; compatibilidad con Playwright en Chromium, Firefox y WebKit y emuladores API 26 y 34; `scripts/iso25010_stats.py` que genere `iso25010.csv` desde los crudos. **Cómo lo verifico:** `python scripts/iso25010_stats.py && git diff --exit-code docs/experimentos/resultados/iso25010.csv`; el CSV incluye Compatibilidad y $n \geq 8$ en todas.

§18 Manuscrito — bien escrito, pero sobre cifras que hay que rehacer

Qué encontré: compila limpio, pero no desde la copia limpia que documenta el README: incluye 16 imágenes de `release/screenshots/` y una de `experiments/.verificacion.json` declara el hash de un PDF anterior (30dd00cd...), no el del corte (03b3a388...). No sigue la estructura de la Tabla 3 (no hay secciones de fundamento teórico, aplicación web, aplicación móvil, pruebas y CI/CD ni observabilidad). Publica el 82,31 % de cobertura filtrada y la atribución falsa de $r = 5$ a la guía; no hay evidencia de i18n ni capturas de `/main`, `/settings` y `/about`; la portada está desfasada (§1). **Cómo debe quedar:** estructura de la Tabla 3; tablas de protocolo, ISO y cobertura incluidas con `\input` desde `.tex` generados por los guiones a partir de los CSV canónicos, de modo que ninguna cifra se teclee; instrucciones de copia limpia completas; hash del PDF actualizado en el mismo commit y comprobado en CI; compilado *después* de la última regeneración de resultados. **Cómo lo verifico:**

```
git archive HEAD docs/entrega4 docs/entrega3/referenciasPFC.bib release/screenshots
↪ experiments/paso8/resultados-reales | tar -x -C /tmp/m
cd /tmp/m/docs/entrega4 && latexmk -pdf -halt-on-error PFC4.tex && grep -c undefined PFC4.log
grep -o "82{,}31" PFC4.tex # vacio
```

§19 Amenazas a la validez y reflexión ética

Qué encontré: el texto está bien hecho, pero describe la campaña actual ($r = 5$, saturación, oráculo retrospectivo) y deberá reescribirse cuando se repita (§15); tres amenazas relevantes (configuración, saturación y carga sintética) no tienen una mitigación explícita, y se cita el código de ética ACM/IEEE-CS de ingeniería de software en lugar del ACM Code of Ethics and Professional Conduct (2018) que pide la guía. **Cómo debe quedar:** amenazas revisadas contra el nuevo diseño, cada una con su frase “Mitigación:” concreta; referencia al código ACM 2018 con al menos dos principios numerados aplicados (1.2, 2.5). **Cómo lo verifico:** `grep -c "Mitigaci" docs/entrega4/PFC4.tex ≥ 11`.

§20 Trabajo en equipo y autoría

Qué encontré: una sola acta (04/09), en la que no figuran Lozano Morales ni Gaibor Rodríguez; una sola aprobación cruzada de PR; 82 commits directos a `main` desde el 01/09. La declaración de IA existe. Aucatoma Celorio tiene el mayor número de commits del equipo (99), pero entre el 12/09 y el corte sus 15 commits son modestos: 292 líneas de pruebas web reales, retirada de `.idea/`, un refactor de complejidad, retoques de README y CI (uno de ellos sólo cambia un texto aunque su mensaje describe otra cosa) y un commit vacío de disparo. **Cómo debe quedar:** un acta por sesión del suspenso con fecha, asistentes, decisiones, responsables y enlace a los PR, versionada el mismo día; cada PR aprobado por un integrante distinto del autor; tabla de autoría por ítem de esta guía en el manuscrito, verificable con `git shortlog`. El aporte del estudiante que rinde el examen se acredita sólo con sus propios commits y PR sobre ítems de esta guía, cuyo mensaje describa exactamente lo que cambia.

4. Orden de ejecución

Cláusula 1 (transferir el repositorio) y §3 protección de `main` → §2 higiene y revocación del secreto → §4 capas → §5 patrones en el checkout (ADR-006 en paralelo) → §6 web y §7 cobertura → §8 push y escáner y §9 → §10 contratos y §12 pruebas → §13 DAG → §14 observabilidad → §15a protocolo (commit) → §15b campaña → §16 paquete → §17 ISO/IEC 25010 (§11 en paralelo desde el principio) → §18 y §19 → §1 portada y §20 → CHANGELOG, manifiestos y, lo último, la etiqueta anotada.

5. Lista de verificación antes de entregar

```
git ls-remote https://github.com/gleiston-guerrero/TiendaTech HEAD      # clausula 1
git cat-file -t v4.1.0 && git rev-parse v4.1.0^{commit} origin/main
grep -rlnE "interface \w*(Payment|Pago)\w*Strategy|interface \w*Observer|\w*Decorator (implements|extends)"
↪ services/*/src/main/java
grep -n "<include>" services/*/pom.xml; grep -n "thresholds" Apps/web/frontend/webapp/vitest.config.ts
python scripts/check_data_checksums.py && python experiments/paso8/campaign_checksums.py
python -c "import csv;print([(x['caracteristica'],x.get('n')) for x in
↪ csv.DictReader(open('docs/experimentos/resultados/iso25010.csv'))])]"
git ls-files | grep -iE "(^|/)\.idea/|\.iml$|\.db-journal$|debug\.apk$"
```

Documento de uso académico. Aplicaciones Distribuidas (ISR-701), UTEQ, 2026–2027 PPA. Verificación ejecutada sobre clon completo del repositorio declarado, con el estado al domingo 13 de septiembre de 2026 a las 19:00, el 16 de septiembre de 2026. Ing. Gleiston Cicerón Guerrero Ulloa, PhD.

Guía de cierre y rúbrica del examen suspenso

Equipo BCEL — AcadTrace, gestión académica escolar con bitácora de auditoría encadenada

Identificación	
Asignatura	Aplicaciones Distribuidas (ISR-701) — 2026–2027 PPA
Paralelo	7.º “A” Software
Repositorio evaluado	gleiston-guerrero/acadtrace
Cabecera evaluada	Rama main, commit 78b0d175 — 720 commits — último commit anterior al domingo 13/09/2026, 19:00; verificación sobre clon completo, 16 de septiembre de 2026
Integrantes	Bedón Viteri Keyla Betzabe · Castro López Pedro Leonardo · Emanuel Pino Juliana Romina · Luna Mora Ernesto Gregory
Rinden el examen suspenso	Los cuatro integrantes
Compleitud del proyecto	40 %
Corte del examen suspenso	Viernes 18 de septiembre de 2026, 23:55 (Ecuador)
Docente	Ing. Gleiston Cicerón Guerrero Ulloa, PhD.

Cláusulas de entrega del examen suspenso (anulan la calificación si no se cumplen)

1. Repositorio transferido y aceptado. El repositorio debe estar transferido a la cuenta del docente (gleiston-guerrero) y la transferencia **aceptada** antes del corte. Si al viernes 18/09/2026 a las 23:55 el repositorio no figura bajo github.com/gleiston-guerrero/, el examen se toma como **no entregado** y su nota es cero, cualquiera que sea el trabajo realizado. Estado al 13/09: al corte el repositorio aún estaba en LE023as/acadtrace; se transfirió después y al 16/09 figura bajo [gleiston-guerrero/acadtrace](https://github.com/gleiston-guerrero/acadtrace). Cumple si sigue así al corte del examen. Todo lo que se califica se revisa en esa URL y en su rama principal; lo que quede en ramas sin fusionar o en cuentas personales no existe para esta evaluación.

2. Carátula en una sola página subida al SGA. Cada estudiante que rinde el examen sube al SGA un único PDF de **una sola página**, sin capturas de cuestionarios, calificaciones ni contenido adicional, con: universidad, facultad y carrera; asignatura, paralelo y periodo; “Examen suspenso — Proyecto Fin de Curso”; nombre completo del estudiante que la sube; equipo y nombre del sistema; integrantes del equipo; docente; URL canónica del repositorio (<https://github.com/gleiston-guerrero/acadtrace>) escrita completa en una línea y como enlace activo; rama, etiqueta anotada de cierre y hash corto del commit final; fecha de entrega. Una carátula de más de una página, con una URL distinta de la canónica o que no se suba al SGA equivale a examen no entregado para ese estudiante.

1. Evaluación de los entregables

#	Entregable / indicador	Estado propio	Estado efectivo	%	Acción
1	Identificación del entregable y URL	Por modificar	Por modificar	70 %	Véase §1
2	Estructura del repositorio, secretos y archivos de raíz	Por culminar	Por culminar	40 %	Véase §2
3	Línea base congelada e integración	Por culminar	Por culminar	25 %	Véase §3
4	Backend en capas y SOLID	Por modificar	Por modificar	50 %	Véase §4
5	Patrones GoF y ADR	Por culminar	Por culminar	30 %	Véase §5
6	Aplicación web funcional	Por modificar	Por modificar	55 %	Véase §6
7	Calidad de la aplicación web	Por culminar	Por culminar	20 %	Véase §7
8	Aplicación móvil funcional	Por modificar	Por modificar	70 %	Véase §8
9	Calidad de la aplicación móvil	Por culminar	Por culminar	35 %	Véase §9
10	Integración y contratos	Por culminar	Por culminar	30 %	Véase §10

#	Entregable / indicador	Estado propio	Estado efectivo	%	Acción
11	Persistencia distribuida	Por culminar	Por culminar	25 %	Véase §11
12	Pirámide de pruebas y cadena de auditoría	Por culminar	Por culminar	35 %	Véase §12
13	Pipeline CI/CD	Por modificar	Por modificar	60 %	Véase §13
14	Observabilidad	Por culminar	Por culminar	40 %	Véase §14
15	Protocolo y campaña experimental	Por culminar	Por culminar	25 %	Véase §15
16	Paquete de datos y reproducibilidad	Por culminar	Por culminar	35 %	Véase §16
17	Evaluación ISO/IEC 25010	Por culminar	Por culminar	20 %	Véase §17
18	Manuscrito, bibliografía y trazabilidad E1–E4	Por modificar	Por culminar	55 %	Véase §18
19	Amenazas a la validez y reflexión ética	Por modificar	Por modificar	50 %	Véase §19
20	Trabajo en equipo y autoría	Por culminar	Por culminar	35 %	Véase §20

Cómo leer las dos columnas de estado

El *estado propio* describe el artefacto en sí. El *estado efectivo* aplica la regla de dependencia: **ningún artefacto puede estar más completo que aquello de lo que se construye**. El backend en capas sostiene a los dos clientes; los clientes y el backend sostienen los contratos y la pirámide de pruebas; las pruebas sostienen el pipeline; el sistema instrumentado sostiene la campaña experimental; la campaña sostiene el paquete de datos; el paquete sostiene la evaluación ISO/IEC 25010; y el manuscrito, las amenazas a la validez y la línea base congelan y reportan todo lo anterior. Las actas, la declaración de uso de IA y los archivos de raíz no propagan: se declaran. Lo que se califica es el estado efectivo.

Lo que está hecho y no se recalifica

Entre el 12/09 y el corte cerraron trabajo real. Los 29 commits que estaban sin fusionar están en `main`. La cabeza de la cadena de auditoría tiene ahora bloqueo efectivo en los tres escritores (`@Lock(PESSIMISTIC_WRITE)` y `FOR UPDATE`) con el reloj de Lamport persistido, y las tres implementaciones de la forma canónica usan los mismos 11 campos: una cadena de cuatro eslabones generada en Java se verifica con el verificador de Python y la manipulación se detecta. Finales de línea normalizados (38 CSV sin CR y comprobación en CI que puede fallar); `npm ci` con lockfiles sincronizados en las cuatro interfaces; sonda de salud del despliegue que propaga el fallo; configuración de gitleaks sin lista blanca; cinco imágenes publicadas en GHCR por commit; Soporte con 73,03 % de instrucciones y compuerta de módulo; seis pruebas Playwright bien diseñadas (fallan sin entorno y verifican escritura y restauración); verificación de bytes 6/6 con diez pruebas que detectan manipulación. La app móvil guarda el JWT en `EncryptedSharedPreferences`, tiene biometría y caché Room sin conexión. El manuscrito compila sin referencias indefinidas.

2. Rúbrica del examen suspenso

Cómo se califica el examen suspenso

Sólo se califican los ítems de esta tabla: lo que está en Hecho no vuelve a puntuar ni se puede perder. El peso de cada ítem sale de sumar cuatro estimaciones de lo que cuesta llevarlo a Hecho con la calidad exigida: **profundidad** (P, 1–5: de aplicar una instrucción a rehacer análisis, diseño experimental o arquitectura con criterio propio), **complejidad** técnica (C, 1–5: de trámite mecánico a coordinar varios servicios, datos y herramientas), **amplitud** (A, 1–5: de un archivo a varios módulos, clientes, flujos y documentos a la vez) y **tiempo** de dedicación (T, 1–5: menos de 2, de 2 a 6, de 6 a 15, de 15 a 30 y más de 30 horas-persona), normalizando

a 100 dentro del equipo. Cada ítem recibe su peso multiplicado por el coeficiente del estado en que lo encuentre el viernes 18 de septiembre a las 23:55: **Hecho 1,00 · Por modificar 0,40 · Por culminar 0,20 · Por iniciar 0,00**. La nota del examen es la suma de puntos dividida entre diez. Un ítem sólo llega a Hecho si la comprobación descrita en “Cómo lo verifico” da el resultado exigido sobre el repositorio del docente; el mensaje de un commit, un archivo vacío o una cifra sin dato crudo que la respalde no cuentan. Las cláusulas de entrega de la primera página y la ausencia total de la aplicación web o de la móvil (factor 0,5) siguen vigentes sobre la nota.

§	Ítem que se califica	P	C	A	T	Pts	Peso %	Estado 13/09	Puntos
§1	URL canónica en README, CITATION.cff, manuscrito, OpenAPI y despliegue; integrantes, correos y roles	1	1	2	1	5	1,8	Por modificar	0,72
§2a	Credenciales y datos personales fuera del árbol y del historial, claves rotadas, sin valores de reserva ni acceso dev-token, escaneo de secretos como compuerta	3	3	3	3	12	4,3	Por culminar	0,86
§2b	Sin informes de cobertura, logs, fotos, APK duplicados ni lockfile huérfano; un solo protocolo, juego de resultados y tablero	1	1	3	2	7	2,5	Por modificar	1,00
§2c	.env.example alineado con compose, propiedades y SPA	1	1	1	1	4	1,4	Por modificar	0,56
§3	Rama principal protegida, PR aprobados por otro integrante, tabla de revisión real, CHANGELOG.md y etiqueta anotada de cierre	2	1	2	2	7	2,5	Por culminar	0,50
§4	Dominio sin JPA ni Spring, puertos en domain/port y aplicación dependiente sólo de puertos en Principal y Secretaría; ArchUnit	4	4	5	5	18	6,6	Por modificar	2,64
§5	Facade, Template Method, Observer y Strategy invocados desde flujos reales con pruebas; ADR-005 y ADR-006 veraces	3	3	3	4	13	4,7	Por culminar	0,94
§6	SPA con /main, sesión fuera de localStorage, validación de expiración, llamadas sólo por el gateway con autorización y portal de directivo	3	3	4	4	14	5,1	Por modificar	2,04
§7	TypeScript estricto, ESLint en CI sin errores, Vitest con cobertura ≥ 70 % y Dockerfile multi-stage para cada SPA	3	3	4	4	14	5,1	Por culminar	1,02
§8	Actualización por arrastre, push demostrable de extremo a extremo y URL base por BuildConfig con HTTPS	2	2	2	2	8	2,9	Por modificar	1,16
§9	Pruebas de los ViewModels, E2E instrumentada en CI y APK release firmado y publicado por CI	3	3	3	3	12	4,3	Por culminar	0,86
§10	Contratos generados por la SPA y la app Kotlin, verificados contra Principal y Docente con estados reales; API de calificaciones autenticada	3	4	4	4	15	5,4	Por culminar	1,08

§	Ítem que se califica	P	C	A	T	Pts	Peso %	Estado 13/09	Puntos
§11	Clúster de base de datos reproducible con healthchecks, Flyway con validación, usuario de ejecución sin privilegios y caída de nodo medida	4	4	3	4	15	5,4	Por culminar	1,08
§12a	Cadena de auditoría: vectores canónicos compartidos, restricción UNIQUE, prueba multiservicio real y fallos que no se pierdan en silencio	5	4	4	4	17	6,2	Por modificar	2,48
§12b	Cobertura $\geq 70\%$ por módulo sin clases elegidas, E2E en verde como compuerta y Testcontainers con migraciones completas	3	4	5	5	17	6,2	Por culminar	1,24
§12c	Rampa 0→200 real, un juego canónico de carga, cifras inexistentes retiradas y comandos del README ejecutables	3	2	3	3	11	4,0	Por culminar	0,80
§13	Compuertas en los needs del despliegue, jobs que hacen lo que su nombre dice, despliegue por imagen y CI sin secretos de producción	3	3	2	3	11	4,0	Por modificar	1,60
§14	Métricas de negocio en flujos reales, registros JSON con trace_id en los cuatro servicios, OpenTelemetry con Collector y tablero único con evidencia	3	3	4	4	14	5,1	Por culminar	1,02
§15	Protocolo con H0/H1 y umbrales versionado antes de medir y campaña medida sobre el sistema real (no por fórmula) con $r = 10$	4	3	3	4	14	5,1	Por culminar	1,02
§16	Generación determinista que reproduce los datos publicados, verificador corregido, manifiesto completo, diccionario y regeneración en CI	4	3	3	3	13	4,7	Por culminar	0,94
§17	ISO/IEC 25010 con las cinco características de la guía, réplicas reales, IC95 % y tabla generada	4	2	2	3	11	4,0	Por culminar	0,80
§18	Manuscrito sin afirmaciones que el código no respalde, PDF igual a la fuente, ≥ 12 referencias citadas con DOI/ISBN y trazabilidad E1→E4	3	2	3	3	11	4,0	Por culminar	0,80
§19	Amenazas con mitigaciones verificables (incluidos datos sintéticos) y reflexión ética $\geq \frac{1}{2}$ página con ACM 2018	2	1	1	2	6	2,2	Por modificar	0,88
§20	Actas contemporáneas sin anacronismos, declaración de IA única, identidades git correctas y aporte propio de cada integrante	2	1	2	2	7	2,5	Por culminar	0,50
Total							100		

Lectura de la rúbrica al 13/09/2026, 19:00

La columna *Estado* es el que encontré en el commit de corte; la columna *Puntos* es peso por coeficiente. Nota provisional del examen suspenso: **2,65 / 10,00**. Si cierran todos los ítems en Hecho, la nota es 10,00. Los cuatro integrantes rinden el examen: el aporte de cada uno se acredita sólo con sus propios commits y PR sobre ítems de esta guía (§20).

3. Trabajo exigido

§1 Identificación del entregable y URL

Qué encontré: la URL canónica no aparece en ningún archivo; LE023as/acadtrace figura en nueve lugares (README, CITATION.cff, portada y cuerpo del manuscrito, docs/api/openapi.yaml, el documento de rotación de secretos, docs/bitacora.md, que además apunta a otro repositorio, y el job de despliegue, que sigue clonando el repositorio antiguo). CITATION.cff empieza con BOM, tiene un correo de autor distinto del institucional y declara version: 4.0.0 sin etiqueta que le corresponda. El README no tiene integrantes ni roles. **Cómo debe quedar:** <https://github.com/gleiston-guerrero/acadtrace> en todos esos lugares; los cuatro integrantes con nombre completo, correo institucional y rol en README y CITATION.cff; versión igual a la etiqueta de cierre. **Cómo lo verifico:** `git grep -n "LE023as/" <etiqueta> vacío; head -c3 CITATION.cff | xxd sin efbbbf.`

§2 Estructura — credenciales, datos personales y basura versionada

a) Secretos y datos sensibles (lo más urgente de esta guía). Corrigieron la lista blanca de gitleaks y los \${VAR} de Principal y Soporte, pero al corte:

- docs/pdf/MANUAL_MAESTRO_COMANDOS_OPERACIONES_SGA.pdf contiene, en texto extraíble, la contraseña de PostgreSQL, la dirección del servidor, el nombre de la clave SSH y cuentas de usuario con contraseña; ningún detector de texto inspecciona PDF.
- El historial no se purgó: los valores reales del secreto JWT, la contraseña de base de datos, la clave AES y el testigo gRPC siguen en decenas de commits alcanzables.
- La migración V18 (13/09) crea el rol sga_app con una contraseña literal, repetida en dos pruebas.
- microservicio-secretaria/.../application.properties define valores de reserva para JWT, AES, base de datos y gRPC: si falta la variable, el servicio firma testigos con una clave conocida; y docker-compose.yml perdió los :? obligatorios.
- JwtAuthFilter.java de Secretaría concede los roles SECRETARIA y DIRECTOR a **cualquier** testigo que empiece por dev-token, y la interfaz de Secretaría tiene un botón visible “Ingresar con los 4 Roles” que lo usa, sin condición de entorno de desarrollo.
- scripts/backups/backup_pre_seed_20260806_135036.sql contiene 677 registros de estudiantes con cédula, nombres y correo, y 21 hashes de contraseña; el 92 % de las cédulas supera el dígito verificador. Mientras no se demuestre que son sintéticos, deben tratarse como datos personales de menores.
- El job secret-scan sólo revisa el rango del evento y ningún job depende de él.

Cómo debe quedar: ni credenciales ni datos personales en el árbol (incluidos PDF, PPTX y SQL) ni en el historial (git filter-repo coordinado), o bien declaración con evidencia fechada (sin los valores) de que todas las credenciales publicadas se rotaron; ningún valor de reserva para secretos; V18 sin contraseña literal (marcador de Flyway); la rama startsWith("dev-token") y el botón eliminados; gitleaks detect --log-opts="--all" como compuerta en los needs de la publicación. **Cómo lo verifico:**

```
gitleaks detect --source . --log-opts="--all" # 0 hallazgos
git grep -nI -E "PASSWORD '[^$]|dev-token" -- ':!*.*md' # vacío
for f in $(git ls-files '*.pdf'); do pdftotext "$f" - | grep -HiE "pass:|password|.pem"; done # vacío
git ls-files scripts/backups # vacío
```

y una petición con Authorization: Bearer dev-token-x a Secretaría devuelve 401.

b) Basura versionada. 823 informes HTML de cobertura (13 MB) y otra copia en Secretaría; build.log, maven-debug.log y otros logs pese a que .gitignore ya los excluye; fotos subidas; dos

APK de depuración; un `package-lock.json` de 102 bytes sin `package.json`; 18 rutas de evidencias con espacios; dos protocolos casi idénticos, dos carpetas de resultados y dos tableros distintos. **Deben:** sacarlos del índice, publicar informes como artefactos de CI y dejar un solo protocolo, un solo juego de resultados y un solo tablero. **c)** `.env.example`. Compose exige `GRAFANA_ADMIN_PASSWORD` y la plantilla define otra variable, así que `docker compose up` falla con la plantilla copiada; faltan `MAIL_PASSWORD`, `FIREBASE_CREDENTIALS_PATH`, `ZIPKIN_ENDPOINT`, `CORS_ORIGIN`, `DB_SSL`, `UPLOADS_PATH`, `VITE_API_URL` y `VITE_API_PRINCIPAL`, y `DB_HOST` apunta al servidor real. **Cómo lo verifico:** `git ls-files | grep -cE "\.log$|docs/cobertura/uploads/"` da 0; `cp .env.example .env && docker compose config -q` termina con 0.

§3 Línea base — revisión cruzada que no existió

Qué encontré: `pre-e4` es ligera y `v1.0.0` es ligera y posterior al corte; no hay `CHANGELOG`. Los 26 PR fusionados entre el 12 y el 13/09 tienen **cero revisiones**, la mayoría fusionados por su autor; los PR que el manuscrito presenta como “validados por otro integrante” (#48, #51, #55, #58, #59, #60) también tienen cero, y el #58 lo abrió y fusionó una persona distinta de la que la tabla indica. No existe ningún issue, aunque el manuscrito describe un flujo “formalizado mediante un issue”. 184 de 271 commits de la primera línea de `main` entraron directamente. Una fusión sin revisión (62933d72, PR #98) dejó el `application.properties` de Secretaría en 0 bytes durante unas 17 horas. **Cómo debe quedar:** protección de `main` (PR obligatorio, una aprobación de otro integrante, verificaciones obligatorias); tabla de revisión cruzada del manuscrito sustituida por los datos reales de la API (PR, autor, quien fusiona, quien aprueba) o eliminada, igual que la descripción del flujo por issues; ramas residuales borradas; `CHANGELOG.md` con entradas fechadas E1–E4 y examen suspenso; etiqueta anotada de cierre sobre el último commit de `main`, creada **al final de todo**. **Cómo lo verifico:**

```
git cat-file -t v4.0.0 && test "$(git rev-parse v4.0.0~{commit})" = "$(git rev-parse origin/main)"
git ls-remote --tags origin v4.0.0 # deben salir DOS líneas
```

y `/pulls/<n>/reviews` con `APPROVED` de un usuario distinto del autor en cada PR del suspenso.

§4 Backend en capas — el dominio de Principal es JPA

Qué encontré: en `sga-principal`, 27 de 74 archivos de `domain` llevan `@Entity`, `@Table` y `@Column` (y dos estrategias llevan `@Component`); no hay ningún puerto en el dominio; 17 archivos de `application` importan `infrastructure` y los servicios usan directamente 25 `JpaRepository`. Entre el 12/09 y el corte se añadió otra entidad JPA al dominio (`EstadoCadenaAuditoria`). En Secretaría el “dominio” son sólo DTO y 15 servicios usan `JdbcTemplate` directamente; Docente (Django) no tiene capa de dominio. Sólo Soporte tiene un puerto con su adaptador. **Cómo debe quedar:** en Principal y Secretaría, dominio sin `jakarta.persistence` ni `org.springframework`, puertos en `domain/port`, entidades JPA y adaptadores con mapeadores en `infrastructure/persistence`, aplicación que dependa sólo de puertos y prueba `ArchUnit`; en Docente, capa de servicios con puertos explícitos y un ADR que justifique la adaptación a Django. **Cómo lo verifico:**

```
grep -rLE "jakarta.persistence|org.springframework" sga-principal/src/main/java/ec/edu/uteq/sga/domain | wc -l
↪ # 0
grep -rL "import ec.edu.uteq.sga.infrastructure" sga-principal/src/main/java/ec/edu/uteq/sga/application | wc
↪ -1 # 0
```

§5 Patrones GoF — cuatro de cinco son clases que nadie llama

Qué encontré: sólo `Repository` se usa. `PortalAcademicoFacade` tiene **cero** llamadores; `ReporteNotasPeriodoPDF` (`Template Method`) tiene cero y genera texto, no PDF; `NotaPublicadaEvent` sólo lo publica la fachada muerta y su oyente escribe en el log “Simulación del despacho” (el push real de Docente no sigue el patrón); `CalculoPromedioStrategy` sólo la usa la fachada muerta,

y el promedio real está fijo (70/30) en `microservicio-docente/docentes/services.py`. ADR-005 cita `CalificacionService` y `CalificacionRepository`, que no existen, no tiene alternativas ni consecuencias negativas y es anterior a las clases. ADR-006 está desactualizado (dice que no se integra FCM y cita 36 pruebas cuando hay 62) y sus criterios no tienen fuente externa. **Cómo debe quedar:** el registro real de calificaciones pasa por la fachada desde un controlador; el promedio se calcula con la estrategia seleccionada por la configuración del año lectivo; un endpoint de reporte por período usa la plantilla y genera PDF; `NotaPublicadaEvent` se publica desde el registro real y su oyente invoca el servicio de push; una prueba de integración por patrón; ADR-005 con rutas verificables, alternativas y consecuencias negativas; ADR-006 actualizado con dos criterios cuantitativos con fuente. **Cómo lo verifico:**

```
for c in PortalAcademicoFacade ReporteNotasPeriodoPDF CalculoPromedioStrategy NotaPublicadaEvent; do
  echo $c $(grep -rl $c sga-principal/src/main/java | grep -v "/$c.java" | grep -ci "controller\|service");
  ↪ done # >= 1 cada uno
```

§6 Aplicación web — el testigo en `localStorage` y un manuscrito que dice lo contrario

Qué encontré: existen `/`, `/login`, `/settings` y `/about`, `i18n` y tema en `sga-frontend` y Secretaría; **falta** `/main`. Las cuatro SPA guardan el JWT en `localStorage` plano, lo que la guía prohíbe expresamente, y `ProtectedRoute` sólo comprueba que exista un testigo, sin validar `exp`; el manuscrito afirma “sin `localStorage`, cookies `HttpOnly` y `SameSite`” y “React + TypeScript” (hay 33 `.jsx`, ningún `.ts` ni `tsconfig`). `Calificaciones.jsx` envía las notas de los estudiantes por HTTP a una dirección fija del servicio de IA, saltando el gateway y **sin cabecera de autorización**; `Reportes.jsx` de Secretaría llama a `localhost:3000`. No hay portal de directivo propio. **Cómo debe quedar:** una SPA canónica con las cinco rutas; testigo en memoria con renovación, `sessionStorage` con expiración o cookie `HttpOnly` emitida por el backend; guarda que valide `exp`; todas las llamadas por el gateway con autorización; vista de directivo con reporte por período; el manuscrito describiendo exactamente eso. **Cómo lo verifico:** `grep -rn "localStorage.*token" sga-principal/sga-frontend/src | wc -l` da 0; `grep -n 'path="/main"' sga-principal/sga-frontend/src/App.jsx`; ninguna dirección IP ni `localhost` en el código de las SPA.

§7 Calidad web — sin tipos, sin lint en CI y sin una sola prueba ejecutada

Qué encontré: las cuatro interfaces son JavaScript puro; `npx eslint .` en `sga-frontend` da 54 errores y 9 avisos, y el job `lint` no ejecuta ESLint (sólo un subconjunto de `flake8`); no hay Vitest ni Jest configurado y el único archivo de prueba web no lo ejecuta ningún script; `test-web` sólo compila. `sga-frontend` y `docente-frontend` no tienen `Dockerfile` y en `compose` corren con `npm run dev`, es decir, el servidor de desarrollo en despliegue. El artefacto `web-dist` por commit sí existe, sin suma. **Cómo debe quedar:** `sga-frontend` en TypeScript con `strict: true` y `tsc --noEmit` en verde; ESLint con cero errores en el job `lint`; Vitest con Testing Library y cobertura ≥ 70 % sobre todo `src/` con `thresholds` que hagan fallar `test-web`; `Dockerfile` multi-stage (Node para construir, `nginx:alpine` para servir) para cada SPA desplegada, usado en `compose` y en la publicación; suma del artefacto `dist`. **Cómo lo verifico:**

```
cd sga-principal/sga-frontend && npm ci && npx tsc --noEmit && npx eslint . && npx vitest run --coverage #
↪ código 0
grep -c "^FROM" sga-principal/sga-frontend/Dockerfile #
↪ >= 2
```

§8 Aplicación móvil

Qué encontré: inicio de sesión restringido a representantes, listado, detalle con calificaciones y asistencia, comunicados, caché `Room`, `SyncWorker`, biometría y servicio FCM registrado. Faltan la actualización por arrastre (sólo hay un botón “Reintentar”); el push no es operativo tal como está el repositorio (sin `google-services.json` y con Firebase deshabilitado en el backend) y no hay evidencia de recepción; la URL base es una dirección fija por HTTP con `usesCleartextTraffic="true"`. **Cómo**

debe quedar: PullToRefreshBox en listado y detalle; instrucciones para suministrar las credenciales de Firebase fuera del repositorio y evidencia fechada de una notificación recibida tras publicar una nota (captura y registro del backend); URL base por BuildConfig con HTTPS o tráfico sin cifrar limitado a depuración. **Cómo lo verifico:** `grep -rn PullToRefresh app-movil-docente/app/src/main;` ninguna dirección IP en `app/src/main`.

§9 Calidad móvil

Qué encontré: 62 pruebas unitarias, pero sólo un ViewModel de 13 está probado (`LoginViewModel` no); la única instrumentada renderiza una pantalla y el CI no la ejecuta. El APK versionado es de depuración, del 04/09, y ni siquiera contiene la mensajería FCM que entró el 09/09; el CI sólo hace `assembleDebug`; al corte no había release firmado. **Cómo debe quedar:** pruebas de los ViewModels de inicio de sesión, listado, detalle y comunicados; una E2E instrumentada con servidor simulado (inicio de sesión, listado, detalle) ejecutada en CI con emulador; APK release firmado con almacén en secretos, generado y publicado por CI con su SHA-256 e idéntico al de `release/apk/`. **Cómo lo verifico:** `grep -n "connectedAndroidTest\\|android-emulator-runner\\|assembleRelease" .github/workflows/ci-cd.yml; apksigner verify --print-certs release/apk/*.apk` sin “Android Debug”.

§10 Contratos — una interacción, con el proveedor simulado

Qué encontré: un único contrato con una interacción (`GET /api/docente/calificaciones/`), cuyo “consumidor” está escrito en Python dentro del propio proveedor y cuya verificación parchea `get_queryset`: sin base de datos ni estados. No hay contrato de la app móvil ni de ninguna SPA, ni contra Principal o Secretaría. `tests/contract/test_contracts.py` son expresiones regulares sobre `.proto` y OpenAPI. Además `CalificacionViewSet` no tiene `permission_classes` ni hay permisos por defecto: **la API de calificaciones de Docente no exige autenticación.** **Cómo debe quedar:** contratos generados por `sga-frontend` (Pact-JS sobre su módulo de servicios) y por la app Kotlin (sobre `RepresentanteApi`), que cubran inicio de sesión, representados, calificaciones y asistencia; verificación contra Principal y Docente con estados que siembren una base de prueba; `IsAuthenticated` en los `ViewSet` con un contrato que cubra el 401; job de contratos en los `needs` de la publicación. **Cómo lo verifico:** `ls pacts/*.json ≥ 3; grep -rn patch.object microservicio-docente/tests/contract` vacío; petición sin testigo a la API de calificaciones devuelve 401.

§11 Persistencia distribuida — no hay clúster

Qué encontré: `docker-compose.yml` define 15 servicios y **ninguno es de base de datos**; todos apuntan a un único PostgreSQL externo. El manuscrito afirma “tres instancias redundantes ...Raft” y particionado por rango, que no existen en el código. La evidencia de tolerancia a fallos mide la conmutación de réplicas de la aplicación detrás de HAProxy, no de la base de datos. Flyway tiene la validación desactivada en `compose`, V13 se modificó después de aplicarse, hay un hueco en V11 y dos `v5_`; Secretaría no ejecuta Flyway. V18 crea `sga_app` con privilegios reducidos, pero **todos los servicios siguen conectándose como** `postgres`, de modo que la revocación no tiene efecto, y la misma migración concede `DELETE` sobre la tabla que guarda la cabeza de la cadena. **Cómo debe quedar:** clúster reproducible en `compose` (tres nodos CockroachDB, o primario y réplicas PostgreSQL) con `healthchecks` y `depends_on: service_healthy`; Flyway con validación activa y cambios de V13 en una migración nueva; usuario de migración separado del de ejecución (`sga_app`) en los tres backends, sin `DELETE` sobre la cadena; guion que detiene un nodo con escrituras en curso y verifica continuidad e integridad de la cadena, con su registro fechado versionado. **Cómo lo verifico:** `grep -c healthcheck docker-compose.yml ≥ 3; grep -rn VALIDATE_ON_MIGRATE=false docker-compose.yml` vacío; ningún `DB_USER` de ejecución igual a `postgres`.

§12 Pruebas y cadena de auditoría

a) Cadena de auditoría. Sigue habiendo tres implementaciones que difieren en casos límite: la clave `contraseña` está escrita en Java con un byte inválido (`"contrase?a"`), de modo que no se excluye y su valor entra en el JSON; `BigDecimal 9.50, 1e20` y las marcas de tiempo se serializan distinto en Java y en Python. Ninguna prueba verifica con Python filas escritas por Java; el reloj vectorial sigue sin efecto; no hay restricción `UNIQUE` que impida la bifurcación; las pruebas de concurrencia no usan las migraciones reales; `guardar` atrapa `Exception` y sólo registra un log, así que **un fallo de auditoría pierde el evento en silencio**; ADR-007 y el documento de alineación contradicen el código. **Deben:** vectores canónicos compartidos (ñ y tildes, decimales, flotantes, fechas, nulos, claves excluidas) que las tres implementaciones pasen en CI; índice único sobre `hash_anterior`; prueba de integración con migraciones reales y escrituras concurrentes desde los tres servicios; fallo de auditoría que aborte la transacción o quede en una cola persistente; ADR-007 reescrito; reloj vectorial con vector recibido real o retirado del diseño.

b) Cobertura, E2E y Testcontainers. Medida sobre todo el código escrito a mano: sga-principal 24,98 % de instrucciones, agregado Java 49,02 %. Las compuertas del 70 % siguen aplicándose a seis y siete clases elegidas a mano (el “90,7 %” del manuscrito sale del agregado de esas siete), y el PR #118 “para alcanzar la meta del 70 %” excluye lógica real (`SecurityConfig`, `GlobalExceptionHandler`, el servicio gRPC escrito a mano) y sube Secretaría de 69,76 % a 70,46 % de líneas. Docente cuenta sus propios archivos de prueba en el denominador. Los HTML de cobertura del 13/09 se editaron a mano en vez de regenerarse. La E2E de navegador estaba **en rojo en el commit de corte** y aun así se desplegó porque no está en los `needs`; modifica calificaciones del entorno desplegado; sus evidencias son de antes de reescribirla. Las suites con contenedores se omiten sin Docker, sólo una aplica migraciones reales y parte de un esquema V1–V8 escrito a mano, y la de Secretaría traga los errores de `migrate()` en un `catch` vacío. **Deben:** regla `BUNDLE` de 70 % (líneas e instrucciones) sin `includes` y excluyendo sólo código generado; pruebas hasta alcanzarla en Principal; `.coveragerc` que omita las pruebas en Docente; un único número por módulo con fecha y comando en README y manuscrito; E2E en verde en el commit de cierre, dentro de los `needs`, contra un entorno de pruebas y con su informe publicado; Testcontainers con migraciones completas que fallen (no se omitan) en CI.

c) Carga y comandos. No hay una rampa válida (el juego “de estrés” llega a 200 usuarios a los 40s y otro se quedó en 1 usuario). El manuscrito conserva filas ISO duplicadas con “0 fallos en 12.265” (cifra que no está en ningún CSV de ningún commit) y “100 % de fallos en 12.735”; cita ocho hashes de commit que no existen; el certificado de Soporte da 0 de 4. Ocho comandos del README fallan en un clon limpio (falta instalación previa, dependencias no declaradas, un `.tex` inexistente, guiones sin permiso de ejecución). **Deben:** rampa con `LoadTestShape 0→200` lineal en 600s; un juego canónico por escenario y los demás en `experimentos/descartados/` con su motivo; cifras y hashes corregidos; README ejecutable de principio a fin. **Cómo lo verifico:**

```
grep -n "<element>CLASS</element>\\|<include>" sga-principal/pom.xml microservicio-secretaria/backend/pom.xml
↪ # vacio
grep -c "12{,}265\\|12{,}735" Informe-E4_BCEL/TA-PFC-E4_BCEL.tex
↪ # 0
grep -rn "contrase?a" --include=*.java . | wc -l
↪ # 0
git ls-files -s scripts/start.sh sga-principal/mvnw app-movil-docente/gradlew | awk '{print $1}' | sort -u
↪ # 100755
```

§13 Pipeline CI/CD — compuertas fuera del camino del despliegue

Qué encontré: sonda de salud efectiva, cinco imágenes por commit e instalación reproducible. Pero `secret-scan` y la E2E no están en ningún `needs` (con la E2E en rojo se desplegó igual) y `build-images` construye en otras ramas aunque fallen las pruebas; `lint` no ejecuta el linter de ninguna SPA, `test-web` no ejecuta pruebas, `integration` es un despliegue por SSH sin pruebas de integración y el APK es sólo de depuración; una fusión reintrodujo una publicación duplicada de

Secretaría; el despliegue reconstruye en el servidor con `docker compose up --build` en lugar de usar las imágenes publicadas; y `test-backend` inyecta la dirección, el usuario `postgres` y la contraseña de la base de datos real. **Cómo debe quedar:** despliegue que dependa de `secret-scan`, E2E, contratos y todas las pruebas; jobs que hacen lo que su nombre dice (§7, §9); integración que levante el stack en el runner, espere salud y ejecute `tests/integration` y contratos antes de desplegar; una sola publicación por imagen, incluidas las SPA; despliegue con `docker compose pull` de las imágenes del commit; ningún secreto de producción en pruebas. **Cómo lo verifico:** lectura de los `needs` de `integration`; `grep -n "DB_HOST" .github/workflows/ci-cd.yml` sin la dirección real; todos los jobs en verde en el commit de cierre.

§14 Observabilidad — métricas de negocio que nadie incrementa

Qué encontré: `SgaBusinessMetrics` define seis métricas `sga_*` sin ningún llamador, y su indicador de estudiantes activos está fijo en 344; Docente sí incrementa las suyas. Registros JSON con `trace_id` sólo en Secretaría y Docente. No hay OpenTelemetry ni Collector (el manuscrito lo deja como trabajo futuro); sólo Soporte envía a Zipkin. Hay dos tableros distintos, ninguno usa `sga_*` ni `docente_*`, y `prometheus.yml` conserva un `remote_write` de plantilla. La captura “bajo carga” es del 31/08, anterior a la corrida oficial y al tablero actual. **Cómo debe quedar:** métricas de negocio incrementadas en flujos reales (nota registrada, matrícula confirmada, notificación enviada, evento de auditoría encadenado) y métricas del motor del clúster; registros JSON con `trace_id` en los cuatro servicios; agente o SDK OpenTelemetry hacia un Collector en `compose` y de ahí a un almacén de trazas; un solo `ops/grafana/pfc-dashboards.json` provisionado con los seis paneles de la guía; captura fechada del tablero y de una traza distribuida durante la corrida canónica.

§15 Campaña experimental — latencias generadas por fórmula

Qué encontré: dos protocolos casi idénticos, sin H0/H1 ni variables, con 30 réplicas en lugar de $r = 10$ con descarte y una cuenta errónea ($30 \times 4 \times 5 = 120$); el manuscrito afirma un descarte de la primera y la última repetición que no existe en el código. Los CSV entraron el 31/08 y los protocolos el 01/09, ya con “métricas obtenidas” dentro: **los umbrales se fijaron después de medir**. En el código del corte, `run_experimentos.py` (modo local, en proceso, sin base de datos ni HTTP) **genera las latencias del experimento 1 con una fórmula** (`base_net = 1.25 + 0.35 * conc` más constantes por mecanismo y ruido) y fija `detectado = False` para M0 y M1, en contradicción con `REPRODUCIBILIDAD.md`. Réplicas degeneradas: 47 filas de `exp1_concurrencia.csv` idénticas salvo el número de réplica, `throughput` 20.000 = 20/0,001, 2 filas distintas de 60 en `exp3`. Los CSV no tienen marcas de tiempo. Ninguna campaña se ejecutó contra los servicios desplegados. **Cómo debe quedar:** un único `experimentos/protocolo-e4.md` con H0/H1, variables, umbrales numéricos, $r = 10$ con descarte implementado y procedimiento de manipulación sobre la base real, versionado en un commit anterior a los datos; campaña en modo HTTP o sobre la base del clúster, con latencias medidas con `perf_counter_ns()` alrededor de escrituras reales, marca de tiempo por réplica y cada réplica como archivo crudo. **Cómo lo verifico:**

```
grep -n "base_net\|detectado = False" experimentos/run_experimentos.py      # vacío
git log --format=%ct -1 -- experimentos/protocolo-e4.md                  # menor que:
git log --format=%ct --diff-filter=A -1 -- experimentos/resultados/raw
```

§16 Paquete de datos — el certificado verifica bytes, no reproducibilidad

Qué encontré: `sha256sum -c` da 6/6, pero regenerando dos veces con el código del corte **los resultados cambian entre ejecuciones** y, salvo `iso25010.csv`, ninguno coincide con el certificado. M3 da `reconciliacion_automatica=False` y el CSV versionado dice `True`. **La detección de M2 y M3 cae del 100 % al 60 %:** las manipulaciones T2 (`payload`) y T5 (marca de tiempo) no se detectan porque el verificador sólo hasha el contenido canónico; ese cambio entró el 13/09 después de los CSV y del certificado. Los datos publicados no salen del código publicado, y puede haber una

brecha real en el verificador. Además el guion escribe fuera de `--output-dir`, el manifiesto cubre 6 archivos (fuera quedan Locust, falsos positivos, HTML y figuras), no hay diccionario de datos, `requirements.txt` sólo declara `matplotlib` y el CI nunca regenera. **Cómo debe quedar:** verificador que incluya `payload` y marca de tiempo (o justificación de por qué T2 y T5 no aplican); generación determinista con semilla en todas las fuentes y figuras sin metadatos variables; escritura sólo en `--output-dir`; un solo juego canónico; manifiesto de todos los crudos; `DICCIONARIO_DATOS.md`; dependencias y versión de Python completas; job de CI que regenere en un directorio temporal y compare con el manifiesto. **Cómo lo verifico:**

```
python experimentos/run_experimentos.py --mode local --output-dir /tmp/a
python experimentos/run_experimentos.py --mode local --output-dir /tmp/b
diff <(cd /tmp/a && sha256sum *) <(cd /tmp/b && sha256sum *)          # sin diferencias
diff <(cd /tmp/a && sha256sum *) <(cd experimentos/resultados && sha256sum $(ls /tmp/a))  # sin diferencias
git status --porcelain docs/                                     # vacío
```

§17 Evaluación ISO/IEC 25010

Qué encontré: la tabla del manuscrito no tiene media, DE ni IC95 %, duplica “Disponibilidad” y “Fiabilidad” con cifras contradictorias, omite Compatibilidad y no mide la tasa de 5xx durante una hora. `iso25010.csv` no contiene réplicas: son 20 ventanas de dos corridas no canónicas (una de ellas con un usuario y 79 % de fallos), con cobertura constante de 30,31 % de un informe antiguo; la tabla usa un juego y el CSV otro. Los intervalos de detección corresponden a $n = 100$ cuando el CSV tiene 150, y la detección real regenerada es del 60 % (§16). **Cómo debe quedar:** las cinco características de la guía con métrica, umbral declarado en el protocolo previo, $n = 10$ réplicas reales tras el descarte, media, DE, IC95 % y contraste explícito con el umbral, calculados por `experimentos/iso25010.py` desde los crudos de §15 y §16 e incluidos en el manuscrito con `\input{tabla_iso.tex}`. **Cómo lo verifico:** `python experimentos/iso25010.py && git diff --exit-code experimentos/resultados/iso25010.csv Informe-E4_BCEL/tabla_iso.tex`.

§18 Manuscrito — afirmaciones que el repositorio desmiente

Qué encontré: compila a 29 páginas sin indefinidas (con 204 cajas desbordadas), pero el PDF versionado es del 12/09 y el `.tex` tiene tres commits posteriores: el PDF aún dice 12,265 cuatro veces, 30.31 y 40.96. Afirma cosas que el código no respalda: React con TypeScript, sesión sin `localStorage` con cookies `HttpOnly`, patrones GoF en flujo real, tres instancias Raft y particionado por rango, un flujo por issues, una tabla de revisión cruzada con validadores que no existen, cobertura del 90,7 % y Soporte en “Node.js Express” (es Spring Boot). Tiene dos secciones de fundamento teórico, filas ISO duplicadas, cifras inferenciales (medianas, U, A12) que no salen de ningún archivo y hashes inexistentes. De 43 entradas bibliográficas sólo se citan 14, de las que 8 tienen DOI y ninguna ISBN (se exigen ≥ 12), con dos pares duplicados y una cita mal atribuida. La matriz de trazabilidad cruza temas con archivos, no la evolución $E1 \rightarrow E4$, y cita rutas inexistentes. **Cómo debe quedar:** ninguna afirmación sin respaldo en el código; cifras incluidas desde tablas generadas por los guiones; ≥ 12 referencias citadas con DOI o ISBN y sin duplicados; sección de trazabilidad $E1 \rightarrow E4$ con commit y ruta de lo que cambió en cada entrega; PDF compilado y versionado en el mismo commit que su `.tex`, *después* de la última regeneración de resultados. **Cómo lo verifico:**

```
test "$(git log -1 --format=%H -- Informe-E4_BCEL/TA-PFC-E4_BCEL.pdf)" = "$(git log -1 --format=%H --
↪ Informe-E4_BCEL/TA-PFC-E4_BCEL.tex)"
pdftotext Informe-E4_BCEL/TA-PFC-E4_BCEL.pdf - | grep -cE "TypeScript|HttpOnly|12,265|90.7"      # 0
grep -c "\\\b\bitem" Informe-E4_BCEL/TA-PFC-E4_BCEL.bbl                                   # >= 12
```

§19 Amenazas a la validez y reflexión ética

Qué encontré: tres internas y dos externas, pero la mitigación por descarte no existe en el código, la de contención de CPU no tiene evidencia, las externas no tienen mitigación y no se trata la

amenaza principal: datos sintéticos y latencias generadas por fórmula. La reflexión ética tiene unas 85 palabras, cita el código ACM/IEEE-CS de 1999 y no aborda los problemas éticos del propio sistema: notas de menores enviadas sin autenticación a un servicio de IA externo, el respaldo con posibles datos personales y las credenciales publicadas. **Cómo debe quedar:** cada amenaza con una mitigación que apunte a un archivo verificable, más la validez de constructo por datos sintéticos; reflexión de al menos media página con el ACM Code of Ethics 2018 (1.3 honestidad, 1.6 privacidad, 2.9 seguridad) aplicado a esas decisiones concretas.

§20 Trabajo en equipo y autoría

Qué encontré: las tres actas, fechadas el 18/08, 28/08 y 03/09, entraron en un solo commit el 04/09 y contienen datos posteriores a su fecha (un tablero creado el 01/09, métricas del 31/08, una mediana de la corrida del 11/09) y afirman haber “auditado y validado” PR con cero revisiones; no hay actas del periodo de correcciones. La declaración de uso de IA existe en tres versiones incompatibles (una de ellas pide que “los demás integrantes completen”). 23 commits llevan el nombre de Emanuel Pino con el correo de Luna Mora, 16 de ellos creados en 37 segundos. Entre la evaluación del 12/09 y el corte, Bedón Viteri hizo el trabajo técnico de más peso (22 commits), Emanuel Pino 11, Castro López 7 cambios pequeños (uno de ellos introdujo los valores de reserva inseguros y otro las exclusiones de cobertura) y Luna Mora 2 (uno de ellos la contraseña literal de V18). **Cómo debe quedar:** nota en las tres actas que reconozca que se redactaron el 04/09 y corrija los anacronismos; actas nuevas versionadas el día de cada reunión del suspenso; una sola declaración de IA coherente entre README y manuscrito; identidades git correctas en cada equipo y aclaración de los commits de identidad mixta; cada ítem de esta guía con un responsable nominal y entregado en PR aprobados por otro integrante. **Como los cuatro rinden el examen, cada uno debe tener aporte técnico propio y verificable:** nadie aprueba con el trabajo de otra persona. **Cómo lo verifico:** `git log --diff-filter=A --format="%h%ad" --date=short -- docs/actas/` frente a la fecha de cada acta; `git shortlog -sne --no-merges 78b0d175..v4.0.0`; revisiones de cada PR en la API.

4. Orden de ejecución

§2a credenciales y datos personales (inmediato: retirar, rotar y purgar antes de cualquier otra reescritura) → §3 protección de `main` → §11 clúster y usuario sin privilegios y §12a cadena de auditoría → §4 capas y §5 patrones (en paralelo) → §6, §8 y §9 clientes → §7, §12b, §10 y §13 pruebas y CI → §14 observabilidad → §15 protocolo (commit) y campaña → §16 reproducibilidad → §17 ISO/IEC 25010 (con §12c rampa) → §18 y §19 con las cifras finales → §1, §2b, §2c y §20 → CHANGELOG, manifiestos y, lo último, la etiqueta anotada.

5. Lista de verificación antes de entregar

```
git ls-remote https://github.com/gleiston-guerrero/acadtrace HEAD
git cat-file -t v4.0.0 && test "$(git rev-parse v4.0.0~{commit})" = "$(git rev-parse origin/main)" && test -f
↪ CHANGELOG.md
gitLeaks detect --source . --log-opts="--all"
for c in PortalAcademicoFacade ReporteNotasPeriodoPDF CalculoPromedioStrategy NotaPublicadaEvent; do echo $c
↪ $(grep -rl $c sga-principal/src/main/java | grep -vc "/$c.java"); done
python experimentos/run_experimentos.py --mode local --output-dir /tmp/a && sha256sum -c
↪ experimentos/SHA256SUMS
grep -n "<element>CLASS</element>\\|<include>" sga-principal/pom.xml microservicio-secretaria/backend/pom.xml
pdfToText Informe-E4_BCEL/TA-PFC-E4_BCEL.pdf - | grep -cE "TypeScript|HttpOnly|12,265"
```

Documento de uso académico. Aplicaciones Distribuidas (ISR-701), UTEQ, 2026–2027 PPA. Verificación ejecutada sobre clon completo del repositorio declarado, con el estado al domingo 13 de septiembre de 2026 a las 19:00, el 16 de septiembre de 2026. Ing. Gleiston Cicerón Guerrero Ulloa, PhD.